

AURIX

Microcontroller Infineon Software Architecture

User's Manual

DIO Driver

Release v8.5

Aug 2016

Edition Aug 2016

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2016 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2016-08-11	8.5	Updated the Deviation in Section 10.1 as per JIRA 0000054991-63 Updated the section 11.4.2 as per JIRA 0000051261-339
2016-05-28	8.4	Updated for Protected mode support changes w.r.t JIRA 0000051261-231 1) Section 2.3 'Software Driver Description'. 2) In Section 7.2 API functions 'Origin' is added and I/O Mode updated. 3) In Section 3 File structure is updated with Dio_Protect.h. 4) In Section 5.3 Details of macros w.r.t protected mode support added.
2015-10-28	8.3	Based on JIRA 0000051108-165, API's in Chapter 7.2 are updated for execution mode: User-1 Mode or Supervisor Mode.
2015-06-25	8.2	Updated Section 8, description for DIO_E_UNINIT Review comments updated(REV-7997) (JIRA 0000052363-71)
2015-04-10	8.1	Updated section 7.2.1, table 69, description w.r.t resetting sfr
2015-03-09	8.0	Updated review comments
2015-03-07	7.1	Updated Section 11, w.r.t TC22x/TC21x
2014-11-14	7.0	Updated the review comments
2014-11-05	6.1	Updated Section 7.2, added I/O access mode for APIs.
2014-09-25	6.0	Baselined for both TC27x BC and CA step devices Updated the review comments,
2014-09-16	5.1	Updated for TC22x, added Section 11.5
2014-08-04	5.0	Updated the Review comments, Added Section 4.9
2014-08-02	4.4	Updated for Section 11 for TC23x, Rephrased the text in Section 5.4.1, 5.4.2 and 5.4.3, to support post-build symbolic name in multi-configuration set.
2014-07-14	4.3	Updated reference section Section 10 : Safety information removed
2014-03-26	4.2	Updated the Reference section. Section 11.1 : wrt DIO149 Section 12: Used AURIX derivatives.
2013-12-06	4.1	Added "SFRs updated" for configuration parameters.
2013-09-02	4.0	Updated with the review comments, REV_004766.
2013-08-22	3.2	Updated with Safety features info, Added Section 10 Safety Information,
2013-06-27	3.1	Updated Section 11, Added TC26x
2013-04-03	3.0	Updated the Review comments, Added CommonPublishedInformation container
2013-03-19	2.3	Updated for TC27xB, Updated references section.
2012-12-19	2.2	Updated the Figure 3 in Section 3
2012-08-02	2.1	Updated section Table 23, section 7.2.1. File Structure.
2012-06-15	2.0	Added the references to AS403
2012-05-15	1.1	Updated wrt AS402

CONFIDENTIAL

Date	Version	Change Description
2011-10-15	1.0	Version Baselined
2011-10-14	0.2	Derived Configuration Parameters are modified at sections 5.5.2, 5.5.3, 5.5.4, 5.5.5. At section 11, Ports availability for TC2Dx has been added.
2011-03-21	0.1	Created initial version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents

Page

1	Introduction	12
1.1	Scope	12
1.2	Abbreviations.....	12
1.3	References	13
2	DIO Driver Overview	14
2.1	Software Hardware Mapping.....	14
2.2	PORT /GPIO hardware features	14
2.3	Software Driver Description	15
3	File Structure	17
4	Getting Started	19
4.1	Read the level of a Dio channel	19
4.2	Change the level of a Dio channel	19
4.3	Toggle the level of a Dio channel.....	20
4.4	Read the level of a Dio Port	21
4.5	Change the level of a Dio Port	22
4.6	Read the level of a Dio Channel Group	22
4.7	Change the level of a Dio Channel Group	23
4.8	Get the version info of Dio driver.....	24
4.9	Use same symbolic name for DioPortChannel configurations.....	24
5	Configuration Documentation	26
5.1	Configuration Concept	26
5.1.1	Configuration Class	26
5.1.2	Configuration Variant	26
5.1.3	How to read the Configuration Class field.....	27
5.2	Configuration Type Definitions	27
5.2.1	Type Definition Dio_PortChannelIdType.....	27
5.2.2	Type Definition Dio_ChannelGroupType	28
5.2.3	Type Definition Dio_ConfigType	28
5.3	Container: DioGeneral	28
5.3.1	DioDevErrorDetect	28
5.3.2	DioVersionInfoApi	28
5.3.3	DioFlipChannelApi	29
5.3.4	DioPBFixedAddress	29
5.3.5	DioUserModelInitApiEnable Enable/Disable	30
5.3.6	DioUserModeRuntimeApiEnable Enable/Disable	30
5.3.7	DioRunningInUser0Mode Enable/Disable	31
5.4	Container: DioConfig.....	31
5.4.1	Container: DioPort.....	32
5.4.1.1	DioPortId	32
5.4.2	Container: DioChannel.....	32
5.4.2.1	DioChannelId	33
5.4.3	Container: DioChannelGroup.....	33
5.4.3.1	DioChannelGroupIdentification	33
5.4.3.2	DioPortMask.....	34
5.4.3.3	DioPortOffset.....	34
5.5	Container: DioSafety	35
5.5.1	DioSafetyEnable	35
5.5.2	DioInitCheckApi.....	35
5.6	Container: CommonPublishedInformation	36
5.6.1	ArMajorVersion.....	36
5.6.2	ArMinorVersion.....	36
5.6.3	ArPatchVersion	36
5.6.4	SwMajorVersion	37
5.6.5	SwMinorVersion	37
5.6.6	SwPatchVersion	37

Table of Contents	Page
5.6.7 ModuleId.....	38
5.6.8 VendorId.....	38
5.6.9 VendorApiInfix.....	38
5.6.10 Release.....	39
5.7 Derived Configuration Parameters.....	39
5.7.1 Dio_kChannelGroupConfig.....	39
5.7.2 DIO_PORTS_AVAILABLE_00_31.....	39
5.7.3 DIO_PORTS_AVAILABLE_32_40.....	40
5.7.4 DIO_PORTS_READONLY_00_31.....	40
5.7.5 DIO_PORTS_READONLY_32_40.....	41
5.7.6 DIO_PORT_<x> MSPIN.....	41
5.7.7 DIO_MASK_UNDEF_PINS_PORT<x>.....	41
5.7.8 DIO_CHANNEL_<Port>_<Pin>.....	42
5.7.9 DIO_PORT_<x>.....	42
5.7.10 Dio_PortIdConfigured.....	42
5.7.11 Dio_ChannelGrpConfigSize.....	43
6 Published parameters.....	44
6.1 VendorId.....	44
6.2 ModuleId.....	44
6.3 InstanceId.....	44
6.4 ARReleaseMajorVersion.....	44
6.5 ARReleaseMinorVersion.....	45
6.6 ARReleaseRevisionVersion.....	45
6.7 SWMajorVersion.....	45
6.8 SWMinorVersion.....	45
6.9 SWPatchVersion.....	46
7 API Documentation.....	47
7.1 API Type Definitions.....	47
7.1.1 Type Definition Dio_ChannelType.....	47
7.1.2 Type Definition Dio_PortType.....	47
7.1.3 Type Definition Dio_ChannelGroupType.....	47
7.1.4 Type Definition Dio_LevelType.....	48
7.1.5 Type Definition Dio_PortLevelType.....	48
7.1.6 Type definition Dio_ConfigType.....	48
7.2 API Functions.....	49
7.2.1 Dio_Init.....	49
7.2.2 Dio_ReadChannel.....	50
7.2.3 Dio_WriteChannel.....	51
7.2.4 Dio_FlipChannel.....	52
7.2.5 Dio_ReadPort.....	54
7.2.6 Dio_WritePort.....	55
7.2.7 Dio_ReadChannelGroup.....	56
7.2.8 Dio_WriteChannelGroup.....	57
7.2.9 Dio_GetVersionInfo.....	58
7.2.10 Dio_InitCheck.....	59
7.3 Interrupt Handling.....	59
8 Error classification.....	60
9 Example Usage.....	61
9.1 Configuration of Driver.....	61
9.1.1 DIO Driver Published Symbolic Names.....	61
9.1.2 Generation of DIO Configuration Data.....	61
9.1.2.1 Configuration of DIO Channel.....	62
9.1.2.2 Configuration of DIO Port.....	62
9.1.2.3 Configuration of DIO Channel Group.....	62
9.2 Initialization of Driver.....	63
9.3 Using the API.....	63

Table of Contents	Page
10 Assumptions and Limitations	64
10.1 Assumptions and Deviations from the Software Specification.....	64
10.2 Limitations and Deviations	64
10.3 Hardware Features Not Supported	64
11 Used AURIX Derivate	65
11.1 TC27x.....	65
11.1.1 TC2Dx	65
11.1.2 TC277.....	66
11.1.3 TC275.....	67
11.2 TC26x.....	68
11.2.1 TC264.....	68
11.2.2 TC265.....	69
11.2.3 TC267.....	70
11.3 TC29x.....	71
11.3.1 TC297.....	71
11.3.2 TC298.....	72
11.3.3 TC299.....	73
11.4 TC23x.....	74
11.4.1 TC233.....	74
11.4.2 TC234.....	75
11.4.3 TC237.....	76
11.5 TC22x/TC21x	77
11.5.1 TC222/TC212.....	77
11.5.2 TC223/TC213.....	79
11.5.3 TC224/TC214.....	80

List of Figures

Page

Figure 1	SW HW Driver Mapping	14
Figure 2	Interfaces of DIO Driver	15
Figure 3	File Structure of DIO Driver	17

List of Tables
Page

Table 1	Abbreviations Used in this Document	12
Table 2	File Contents	17
Table 3	PortPin Configuration	19
Table 4	DioChannel Configuration	19
Table 5	PortPin Configuration	19
Table 6	DioChannel Configuration	20
Table 7	PortPin Configuration	20
Table 8	DioChannel Configuration	20
Table 9	PortPin Configuration	21
Table 10	DioPort Configuration	21
Table 11	PortPin Configuration	22
Table 12	DioPort Configuration	22
Table 13	PortPin Configuration	22
Table 14	DioChannelGroup Configuration	23
Table 15	PortPin Configuration	23
Table 16	DioChannelGroup Configuration	23
Table 17	Dio General Configuration	24
Table 18	Configuration Class Example 1	27
Table 19	Configuration Class Example 2	27
Table 20	Type Definition Dio_PortChannelIdType	27
Table 21	Members of Type definition Dio_PortChannelIdType	27
Table 22	Development Error Detection	28
Table 23	Module Version Information	28
Table 24	DioFlipChannelApi	29
Table 25	DioPBFixedAddress	29
Table 26	Enabling/Disabling SFR resetting at Initialization	30
Table 27	Enabling/Disabling SFR resetting at Initialization	30
Table 28	Enabling/Disabling SFR register access in USER0 mode	31
Table 29	Dio Port Id	32
Table 30	Dio Channel Id	33
Table 31	Dio Channel Group Identification	33
Table 32	Dio Port Mask	34
Table 33	Dio Port Offset	34
Table 34	DioSafetyEnable	35
Table 35	DioInitCheckApi	35
Table 36	ArMajorVersion	36
Table 37	ArMinorVersion	36
Table 38	ArPatchVersion	36
Table 39	SwMajorVersion	37
Table 40	SwMinorVersion	37
Table 41	SwPatchVersion	37
Table 42	ModuleId	38
Table 43	VendorId	38
Table 44	VendorApiInfix	38
Table 45	Release	39
Table 46	Dio_kChannelGroupConfig	39
Table 47	DIO_PORTS_AVAILABLE_00_31	39
Table 48	DIO_PORTS_AVAILABLE_32_40	40
Table 49	DIO_PORTS_READONLY_00_31	40
Table 50	DIO_PORTS_READONLY_32_40	41
Table 51	DIO_PORT_<x> MSPIN	41
Table 52	DIO_MASK_UNDEF_PINS_PORT<x>	41
Table 53	DIO_CHANNEL_<Port>_<Pin>	42
Table 54	DIO_PORT_<x>	42
Table 55	Dio_PortIdConfigured	42
Table 56	Dio_ChannelGrpConfigSize	43
Table 57	VendorId	44
Table 58	ModuleId	44

List of Tables
Page

Table 59	Module instance id	44
Table 60	Dio Driver AUTOSAR Major Version.....	44
Table 61	DIO Driver AUTOSAR Minor Version.....	45
Table 62	Dio Driver AUTOSAR Patch Version	45
Table 63	Dio Driver SW Implementation Major Version	45
Table 64	Dio Driver SW Implementation Minor Version	45
Table 65	Dio SW Implementation Patch Version	46
Table 66	Type Definition: Dio_ChannelType	47
Table 67	Type Definition: Dio_PortType	47
Table 68	Type Definition: Dio_ChannelGroupType	47
Table 69	Type Definition: Dio_LevelType	48
Table 70	Type Definition: Dio_PortLevelType	48
Table 71	Type definition Dio_ConfigType	48
Table 72	Specification of API Dio_Init.....	49
Table 73	Specification of API Dio_ReadChannel	50
Table 74	Specification of API Dio_WriteChannel.....	51
Table 75	Specification of API Dio_FlipChannel	52
Table 76	Specification of API Dio_ReadPort	54
Table 77	Specification of API Dio_WritePort	55
Table 78	Specification of API Dio_ReadChannelGroup	56
Table 79	Specification of API Dio_WriteChannelGroup.....	57
Table 80	Specification of API Dio_GetVersionInfo	58
Table 81	Specification of API Dio_InitCheck.....	59
Table 82	Error Classification for DETs.....	60
Table 83	Error Classification for Safety.....	60
Table 84	Portpin details for TC2Dx.....	65
Table 85	Portpin details for TC277	66
Table 86	Portpin details of TC275.....	67
Table 87	Portpin details for TC264	68
Table 88	Portpin details of TC265.....	69
Table 89	Portpin details for TC267	70
Table 90	Portpin details for TC297	71
Table 91	Portpin details for TC298	72
Table 92	Portpin details for TC299	73
Table 93	Portpin details for TC233	74
Table 94	Portpin details for TC234	75
Table 95	Portpin details for TC237	76
Table 96	PortPin details for TC222/TC212	77
Table 97	PortPin details for TC223/TC213	79
Table 98	PortPin details for TC224/TC214	80

List of Code Listings

Page

Code Listing 1	Programming Sequence for using DIO Driver to read the level of a Dio channel.....	19
Code Listing 2	Programming Sequence for using DIO Driver to change the level of a Dio channel	20
Code Listing 3	Programming Sequence for using DIO Driver to toggle the level of a Dio channel	20
Code Listing 4	Programming Sequence for using DIO Driver to read the level of a Dio port	21
Code Listing 5	Programming Sequence for using DIO Driver to change the level of a Dio Port	22
Code Listing 6	Programming Sequence for using DIO Driver to read the level of a Dio channel group	23
Code Listing 7	Programming Sequence for using DIO Driver to change the level of a Dio Channel Group 23	
Code Listing 8	Programming Sequence for using DIO Driver to get the version info of dio driver	24
Code Listing 9	Programming Sequence for using DIO Driver to use same symbolic name across config sets info of Dio driver	24
Code Listing 10	MCU specific DIO Channel and DIO Port published symbolic names	61
Code Listing 11	Example for DIO Channel configuration	62
Code Listing 12	Example for DIO Port configuration	62
Code Listing 13	Symbolic Names for the DIO CHANNEL GROUPS	62
Code Listing 14	Example for DIO Channel Group Data Initialization	62
Code Listing 15	Using DIO Driver Services.....	63

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the DIO driver.

The DIO driver is built according to the AUTOSAR specification, refer [\[2\]](#).

This User Manual is intended to help the user to get acquainted with the DIO driver implementation for the AURIX microcontrollers. It is the documentation that describes how to use the DIO driver.

1.1 Scope

This document addresses the following features of the DIO driver implementation,

- AURIX microcontroller HW peripherals used to realize the driver
- Safety mechanism to achieve ASIL B (D) for Initialization, actuation and acquisition.
- File structure of DIO driver
- Configuration parameters of the driver
- Parameters published by DIO driver
- Resource usage
- API specification
- Code snippets illustrating the common use cases.
- Limitations and assumptions

This documentation applies to the DIO driver Variant: Variant PB

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 1](#).

Table 1 Abbreviations Used in this Document

Abbreviation	Explanation
μC	Microcontroller
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
BSW	Basic Software
DET	Development Error Tracer
DIO	Digital input/output
DIO Channel	A general-purpose digital IO pin represents a DIO channel.
DIO Channel Group	Several adjoining DIO channels within a DIO port can be combined to a formal logical channel group.
DIO Port	In the DIO Driver it is possible to group several DIO channels by hardware (typically controlled by one hardware register) to represent a DIO port.
DS	Design Specification
GPIO	General Purpose Input Output
HW	Hardware
I/O	Input/output
ID	Identifier
IFX	Infineon Technologies
ISR	Interrupt Service Routine
MCAL	Microcontroller Abstraction Layer

MCU	Microcontroller Unit
SW	Software
TTL	Transistor Transistor Logic
QM	Quality Module
ASIL	Automotive Safety Integrity Level

1.3 References

This section lists down all relevant documents for this User Manual.

Refer to the latest version for the documents without the version information.

- [1] Requirements of the DIO driver: AUTOSAR_SRS_DIO_Driver.pdf , Release 4.0, Rev 3
- [2] Specifications of the DIO driver: AUTOSAR_SWS_DIO_Driver.pdf, Release 4.0, Rev 3
- [3] Target Data Sheet: tc26xB_um_v1.3.pdf, tc29xB_um_v1.2_2.pdf, tc27xC_um_v2.0OM.pdf, tc27xD_um_v2.2.pdf, tc23x_tc22x_um_v1.1.pdf.
- [4] Header file for DIO: Dio.h,
- [5] HIS_recommendation_AUTOSAR.xls
- [6] Port user manual: Aurix_MC-ISAR_UM_PORTDriver.pdf

2 DIO Driver Overview

The DIO driver provides

- Port
- Channel
- Channel group

Based read and write access to the internal general purpose I/O ports. All read and write services in the DIO driver are un-buffered.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the hardware peripherals and their interaction with the software drivers.

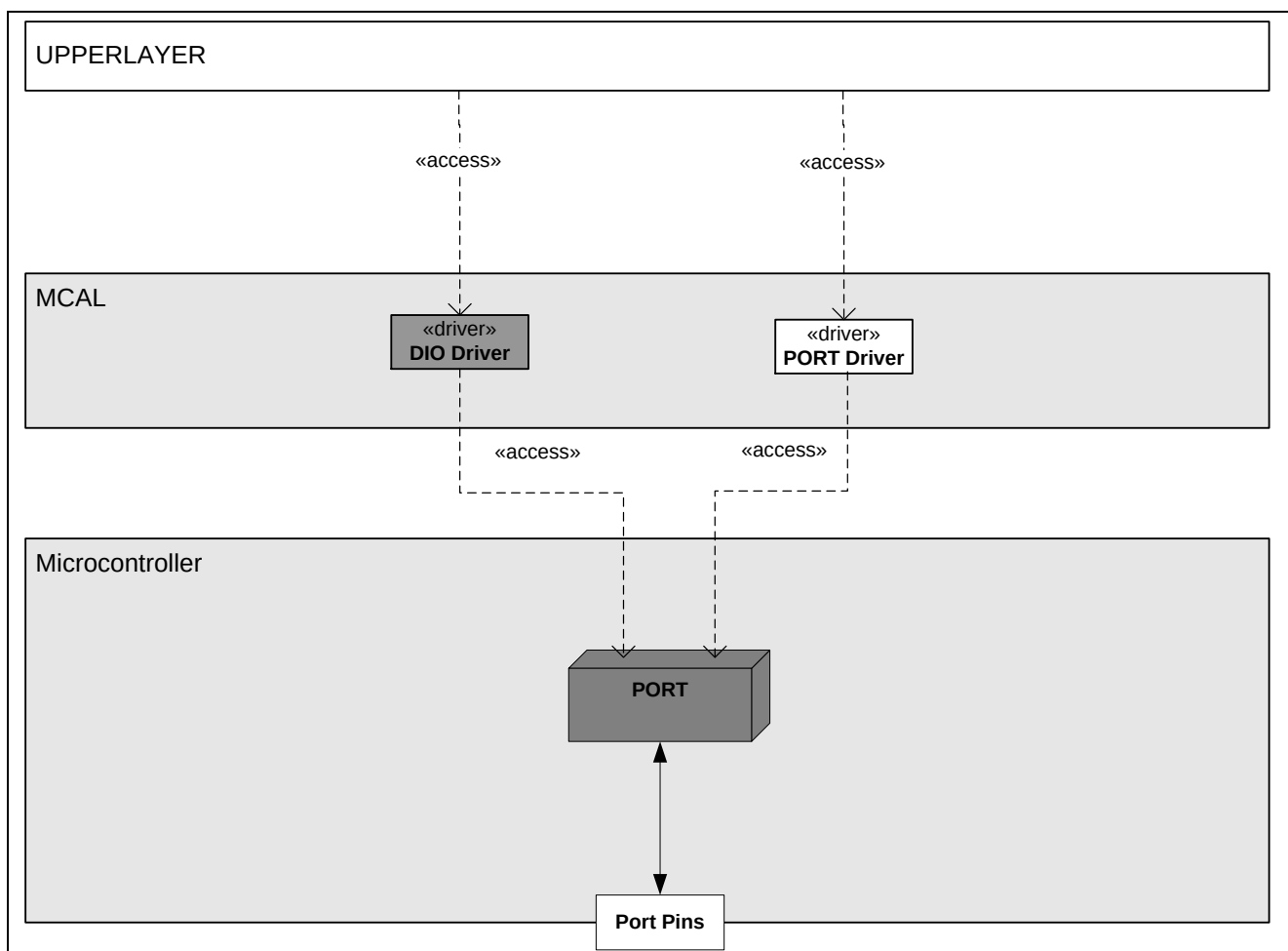


Figure 1 SW HW Driver Mapping

2.2 PORT /GPIO hardware features

DIO, PORT and SPI driver modules use the PORTS HW. The resource conflict has to be handled by the user/integrator.

Refer to Port module's User manual for further details [\[6\]](#).

2.3 Software Driver Description

The **Figure 2** shows the Application Programming Interface (API) for the DIO driver and its interaction with other drivers. Development Error Tracer (DET) is required if DET is enabled for the DIO driver. There are no callback notifications from the DIO driver. The DIO driver provides seven APIs for its user.

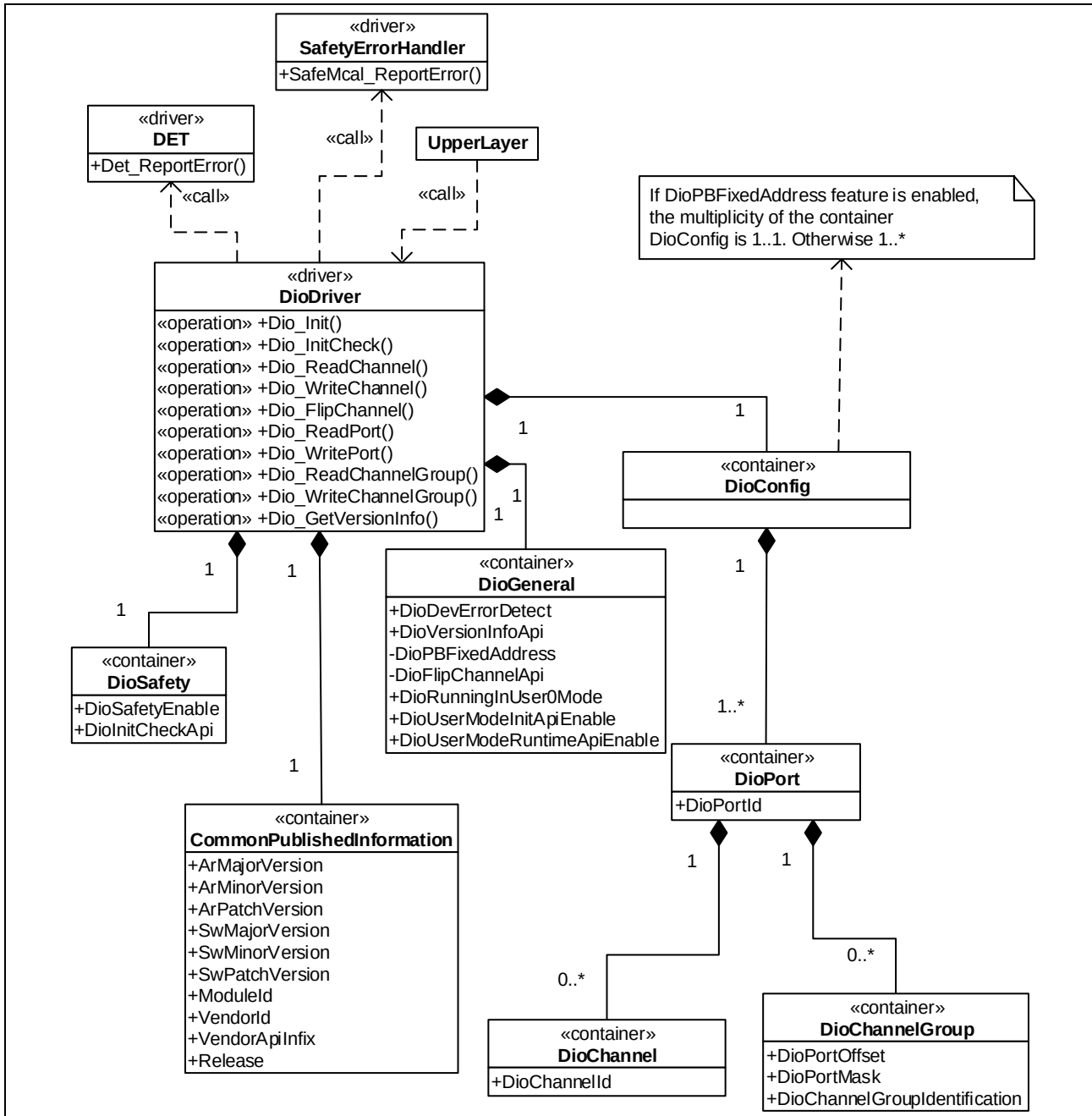


Figure 2 Interfaces of DIO Driver

The configuration is further explained in chapter 4.9. The API's are explained in chapter 7.

The PORT driver configures all the ports and port pins of the microcontroller. The read and write operations on these is provided by DIO driver.

Notes:

- The Port Driver shall be initialized prior to the use of DIO functions. Otherwise DIO functions will exhibit undefined behavior.
- The output data latch of an input channel will be set / cleared so that the required level is output from the pin when the port driver configures the pin as a DIO output pin.

The reconfiguration of the port pin direction and mode during runtime is provided by PORT driver.

In short the DIO driver only provides services for reading from and writing to

- DIO Channels
- DIO Ports
- DIO Channel Groups

DIO Channel: A general-purpose digital I/O pin represents a DIO channel. It may be configured as input or output by the PORT driver.

DIO Port: In the DIO Driver it shall be possible to group several DIO channels by hardware (typically controlled by one hardware register) to represent a DIO port. The single DIO channel levels inside a DIO port represent a bit in the DIO port value, depending on their position inside the port.

DIO Channel Group: Several adjoining DIO channels within a DIO port can be combined to a formal logical channel group.

All read and write services in the DIO driver are un-buffered.

3 File Structure

This section provides details about the DIO driver files and other related files.

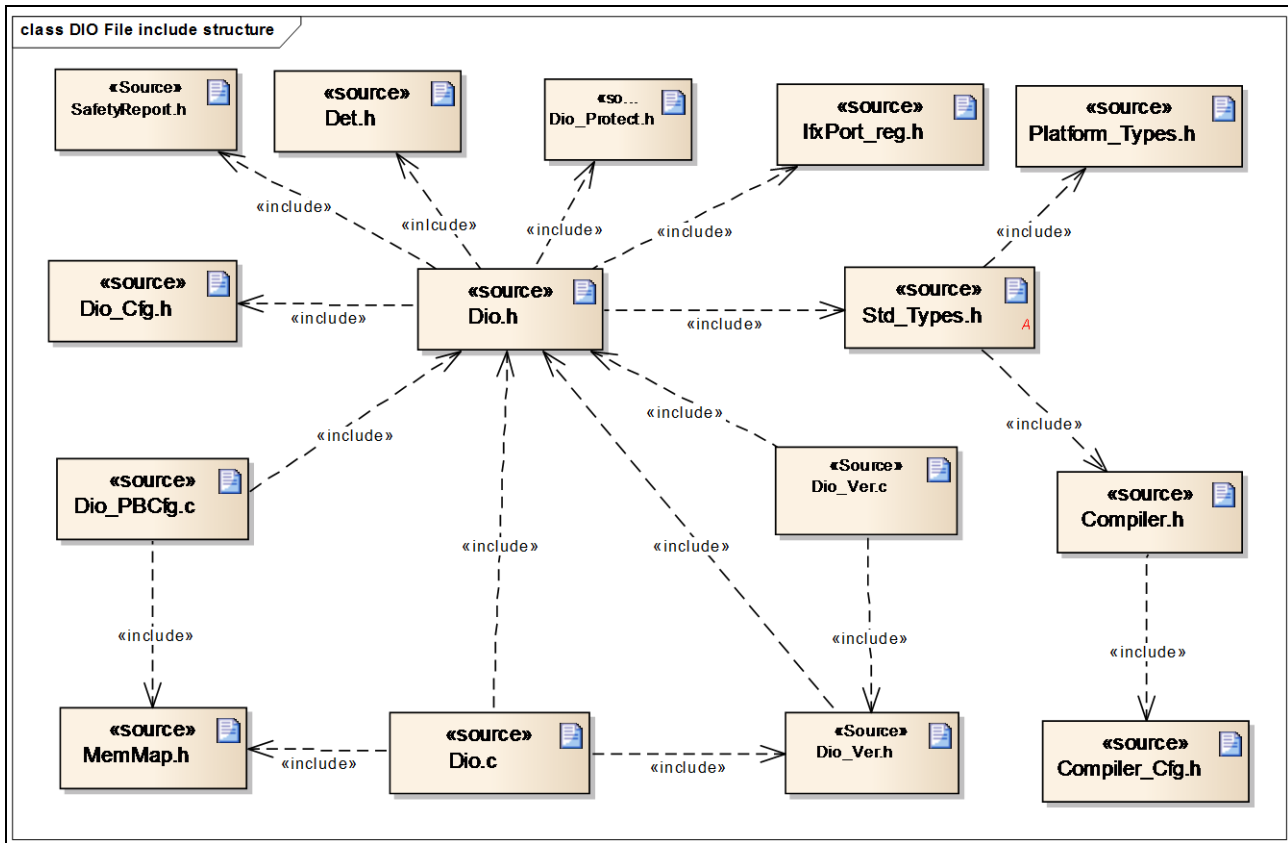


Figure 3 **File Structure of DIO Driver**

Table 2 File Contents

File Name	File Contents
Platform_Types.h	Platform Dependencies are declared here.
Compiler.h	Tasking specific Compiler abstraction is declared here.
Compiler_Cfg.h	MC-ISAR AURIX specific Compiler abstraction is declared here.
Std_Types.h	Standard Data types to be used are declared here.
lfxPort_reg.h	Includes the register definition file for microcontroller.
Mcal.h	Provides library functions for MCAL layer
Det.h	Provides the functionality of Development Error tracer.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections.
Dio.h	Provides the DIO driver functionality
Dio.c	Implementation of DIO driver functionality.
Dio_Cfg.h	Provides Pre Compile configuration data of the DIO driver.
Dio_PBCfg.c	Provides Post Build configuration data of the DIO driver.
Dio_Ver.c	Version specific implementation of DIO driver functionality.
Dio_Ver.h	Header file for Dio_Ver.c

SafetyReport.h	Provides the functionality of Safety error handler
Dio_Protect.h	Included to define all the first level indirection macros for Protected mode support for supervisory access via OS calls.

Note: Dio.h file includes lfxPort_reg.h file, as some of the APIs are realized as inline function.

4 Getting Started

This section gives you a way of quickly programming the DIO driver for typical functionality.

4.1 Read the level of a Dio channel

Configurations are mentioned in Tables 3 and 4; while the sample code is given in Code Listing 1.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 3 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x013
PortPinSymbolicName	PORT_1_PIN_3
PortPinDirection	PORT_PIN_IN
PortPinDirectionChangeable	FALSE
PortPinInputCharacteristic	PORT_PIN_IN_PULL_UP

Table 4 DioChannel Configuration

Configuration Parameter	Configuration Value
DioPortId	0x1
Name	DioPort_1
DioChannelId	0x3
Name	DioChannel_1_3

Code Listing 1 Programming Sequence for using DIO Driver to read the level of a Dio channel

```

001:  /*Variable declaration*/
002:  Dio_LevelType Level;
003:
004:  /* Initialization of PORT Module */
005:  Port_Init(&Port_ConfigRoot[0]);
006:
007:  /* Initialization of DIO Module */
008:  Dio_Init(&Dio_ConfigRoot[0]);
009:
010:  /* Read the level of the Dio Channel */
011:  Level = Dio_ReadChannel(DioChannel_1_3);

```

4.2 Change the level of a Dio channel

Configurations are mentioned in Tables 5 and 6; while the sample code is given in Code Listing 2.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 5 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x020
PortPinSymbolicName	PORT_2_PIN_0

Configuration Parameter	Configuration Value
PortPinDirection	PORT_PIN_OUT
PortPinOutputCharacteristic	PORT_PIN_OUT_PUSH_PULL
PortPinLevelValue	PORT_PIN_LOW

Table 6 DioChannel Configuration

Configuration Parameter	Configuration Value
DioPortId	0x2
Name	DioPort_2
DioChannelId	0x0
Name	DioChannel_2_0

Code Listing 2 Programming Sequence for using DIO Driver to change the level of a Dio channel

```

001:  /* Initialization of PORT Module */
002:  Port_Init(&Port_ConfigRoot[0]);
003:
004:  /* Initialization of DIO Module */
005:  Dio_Init(&Dio_ConfigRoot[0]);
006:
007:  /* Write the level of the Dio Channel */
008:  Dio_WriteChannel(DioChannel_2_0, STD_HIGH);

```

4.3 Toggle the level of a Dio channel

Configurations are mentioned in Tables 7 and 8; while the sample code is given in Code Listing 3.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 7 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x020
PortPinSymbolicName	PORT_2_PIN_0
PortPinDirection	PORT_PIN_OUT
PortPinOutputCharacteristic	PORT_PIN_OUT_PUSH_PULL
PortPinLevelValue	PORT_PIN_LOW

Table 8 DioChannel Configuration

Configuration Parameter	Configuration Value
DioPortId	0x2
Name	DioPort_2
DioChannelId	0x0
Name	DioChannel_2_0

Code Listing 3 Programming Sequence for using DIO Driver to toggle the level of a Dio channel

Code Listing 3 Programming Sequence for using DIO Driver to toggle the level of a Dio channel

```

001:  /*Variable declaration*/
002:  Dio_LevelType Level;
003:  /* Initialization of PORT Module */
004:  Port_Init(&Port_ConfigRoot[0]);
005:
006:  /* Initialization of DIO Module */
007:  Dio_Init(&Dio_ConfigRoot[0]);
008:
009:  /* Toggle the level of a Dio Channel */
010:  Level = Dio_FlipChannel(DioChannel_2_0);

```

4.4 Read the level of a Dio Port

Configurations are mentioned in Tables 9 and 10; while the sample code is given in Code Listing 4.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 9 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x013
PortPinSymbolicName	PORT_1_PIN_3
PortPinDirection	PORT_PIN_IN
PortPinDirectionChangeable	FALSE
PortPinInputCharacteristic	PORT_PIN_IN_PULL_UP

Note:

The above settings need to be done for all the available pins of Port 01. i.e., PortPin 0x014 to 0x017 should be configured as PortPin 0x013.

Table 10 DioPort Configuration

Configuration Parameter	Configuration Value
DioPortId	0x1
Name	DioPort_1

Code Listing 4 Programming Sequence for using DIO Driver to read the level of a Dio port

```

001:  /*Variable declaration*/
002:  Dio_PortLevelType Level;
003:
004:  /* Initialization of PORT Module */
005:  Port_Init(&Port_ConfigRoot[0]);
006:
007:  /* Initialization of DIO Module */
008:  Dio_Init(&Dio_ConfigRoot[0]);
009:
010:  /* Read the level of the Dio Port */
011:  Level = Dio_ReadPort(DioPort_1);

```

4.5 Change the level of a Dio Port

Configurations are mentioned in Tables 11 and 12; while the sample code is given in Code Listing 5.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 11 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x020
PortPinSymbolicName	PORT_2_PIN_0
PortPinDirection	PORT_PIN_OUT
PortPinOutputCharacteristic	PORT_PIN_OUT_PUSH_PULL
PortPinLevelValue	PORT_PIN_LOW

Note:

The above settings need to be done for all the available pins of Port 02. i.e., PortPin 0x021 to 0x02B should be configured as PortPin 0x020.

Table 12 DioPort Configuration

Configuration Parameter	Configuration Value
DioPortId	0x2
Name	DioPort_2

Code Listing 5 Programming Sequence for using DIO Driver to change the level of a Dio Port

```

001:  /* Initialization of PORT Module */
002:  Port_Init(&Port_ConfigRoot[0]);
003:
004:  /* Initialization of DIO Module */
005:  Dio_Init(&Dio_ConfigRoot[0]);
006:
007:  /* Write the level of the Dio Port */
008:  Dio_WritePort(DioPort_2, 0x0A0A);

```

4.6 Read the level of a Dio Channel Group

Configurations are mentioned in Tables 13 and 14; while the sample code is given in Code Listing 6.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 13 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x013
PortPinSymbolicName	PORT_1_PIN_3
PortPinDirection	PORT_PIN_IN
PortPinDirectionChangeable	FALSE
PortPinInputCharacteristic	PORT_PIN_IN_PULL_UP

Note:

The above settings need to be done for all the available pins of Port 01. i.e., PortPin 0x014 to 0x017 should be configured as PortPin 0x013.

Table 14 DioChannelGroup Configuration

Configuration Parameter	Configuration Value
DioPortId	0x1
Name	DioPort_1
DioChannelGroupIdentification	DioChGrpId_P1_0
DioPortMask	0x78

Code Listing 6 Programming Sequence for using DIO Driver to read the level of a Dio channel group

```

009:  /*Variable declaration*/
010:  Dio_PortLevelType Level;
011:
012:  /* Initialization of PORT Module */
013:  Port_Init(&Port_ConfigRoot[0]);
014:
015:  /* Initialization of DIO Module */
016:  Dio_Init(&Dio_ConfigRoot[0]);
017:
018:  /* Read the level of the Dio Channel Group */
019:  Level = Dio_ReadChannelGroup(DioChGrpId_P1_0);

```

4.7 Change the level of a Dio Channel Group

Configurations are mentioned in Tables 15 and 16; while the sample code is given in Code Listing 7.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 15 PortPin Configuration

Configuration Parameter	Configuration Value
PortPinId	0x020
PortPinSymbolicName	PORT_2_PIN_0
PortPinDirection	PORT_PIN_OUT
PortPinOutputCharacteristic	PORT_PIN_OUT_PUSH_PULL
PortPinLevelValue	PORT_PIN_LOW

Note:

The above settings need to be done for all the available pins of Port 02. i.e., PortPin 0x021 to 0x02B should be configured as PortPin 0x020.

Table 16 DioChannelGroup Configuration

Configuration Parameter	Configuration Value
DioPortId	0x2
Name	DioPort_2
DioChannelGroupIdentification	DioChGrpId_P2_0
DioPortMask	0x3C

Code Listing 7 Programming Sequence for using DIO Driver to change the level of a Dio Channel Group

Code Listing 7 Programming Sequence for using DIO Driver to change the level of a Dio Channel Group

```
020:  /* Initialization of PORT Module */
021:  Port_Init(&Port_ConfigRoot[0]);
022:
023:  /* Initialization of DIO Module */
024:  Dio_Init(&Dio_ConfigRoot[0]);
025:
026:  /* Write the level of the Dio Channel Group */
027:  Dio_WriteChannelGroup(DioChGrpId_P2_0);
```

4.8 Get the version info of Dio driver

Configurations are mentioned in Tables 17; while the sample code is given in Code Listing 8.

The configuration tables describe the modifications required; other values can be left at defaults.

Table 17 Dio General Configuration

Configuration Parameter	Configuration Value
DioVersionInfoApi	DIO_VERSION_INFO_API = STD_ON

Code Listing 8 Programming Sequence for using DIO Driver to get the version info of dio driver

```
001:  /* variable declaration*/
002:  /* Pointer to store Version Info of Port module */
003:  Std_VersionInfoType VersionInfo;
004:
005:  /* Initialization of PORT Module */
006:  Port_Init(&Port_ConfigRoot[0]);
007:
008:  /* Initialization of DIO Module */
009:  Dio_Init(&Dio_ConfigRoot[0]);
010:
011:  /* Get the Version Information */
012:  Dio_GetVersionInfo(&VersionInfo);
```

4.9 Use same symbolic name for DioPortChannel configurations

This feature is not applicable for TC27XBC release.

Configurations are mentioned in Tables 17; while the sample code is given in Code Listing 8.

The configuration tables describe the modifications required; other values can be left at defaults.

Configure the channels in the configuration tool as below,

Config1:

Define DioChannelLED name and assign pin 33.8.

Config2:

Define DioChannelLED name and assign pin 33.9.

Code Listing 9 Programming Sequence for using DIO Driver to use same symbolic name across config sets info of Dio driver

Code Listing 9 Programming Sequence for using DIO Driver to use same symbolic name across config sets info of Dio driver

```
013:  /* Initialization of PORT Module */
014:  Port_Init(&Port_ConfigRoot[0]);
015:
016:  /* Initialization of DIO Module */
017:  Dio_Init(&Dio_ConfigRoot[0]);
018:
019:  /* Toggles Pin 33.8 */
020:  Dio_FlipChannel(DioChannelLED)
021:
022:  Dio_Init(&Dio_ConfigRoot[1]);
023:
024:  /* Toggles Pin 33.9 */
025:  Dio_FlipChannel(DioChannelLED);
```

Note: Similar approach can be followed for symbolic names with respect to DioPort and DioChannelGroup.

5 Configuration Documentation

The following chapters summarize all configuration parameters, their description and the type definition used to represent them in the code. This includes those specified in the AUTOSAR DIO specification and the microcontroller specific parameters.

The DIO driver works on the ports and port pins that are configured using PORT driver.

The configuration of DioPort, DioChannel and DioChannelGroup is referenced via symbolic names through the file Dio_Cfg.h. The published MCU specific DIO symbolic names are in the Dio_Cfg.h. The file that contains the instantiation (=definition) of the DIO channel group configuration structure includes Dio_Cfg.h and uses the defined values for initialization of structure elements. Refer AUTOSAR DIO Driver specification [2] and configuration example.

5.1 Configuration Concept

5.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below,

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be frozen before compilation and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

5.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

The DIO Driver is delivered as a Variant PB

5.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type. Topics **5.1.1** , **5.1.2** should be read for understanding the configuration class field.

Table 18 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, and Variant PC.

Table 19 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a,

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

5.2 Configuration Type Definitions

The following type definitions are used for the purpose of defining configuration data.

5.2.1 Type Definition Dio_PortChannelIdType

Table 20 Type Definition Dio_PortChannelIdType

Syntax:	Dio_PortChannelIdType
Type:	struct
File:	Dio.h
Range:	Refer Table 21
Description:	This structure is used to identify the DioPort and DioChannel are configured for the current configuration.

Table 21 Members of Type definition Dio_PortChannelIdType

Parameter name	Type/Range	Description
Dio_PortIdConfig	uint32	Gives whether the Dio port is configured or not.
Dio_ChannelConfig	uint32	Gives the Channel Id of the configured channels per port. One bit per pin, Bit 0 for Pin0 and Bit15 for Pin 15. 1 indicates the pin configured 0 indicates pin is not configured.

5.2.2 Type Definition Dio_ChannelGroupType

Refer [7.1.3](#)

5.2.3 Type Definition Dio_ConfigType

Refer [7.1.6](#).

5.3 Container: DioGeneral

This container lists down all the general configuration parameters for DIO driver.

Its multiplicity is 1.

5.3.1 DioDevErrorDetect

Table 22 Development Error Detection

Name:	DioDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_Cfg.h		
Range:	TRUE - DioDevErrorDetect is Enabled FALSE - DioDevErrorDetect is Disabled		
Default Value:	FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Switches the Development Error Detection and Notification STD_ON or STD_OFF. This configuration parameter is named as DIO_DEV_ERROR_DETECT in the configuration file.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.3.2 DioVersionInfoApi

Table 23 Module Version Information

Name:	DioVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_Cfg.h		
Range:	TRUE - Version info API enabled FALSE - Version Info API disabled		
Default Value:	FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

Description:	Pre-processor switch STD_ON / STD_OFF used to enable / disable the API to read out the modules version information, i.e. Dio_GetVersionInfo() This configuration parameter is named as DIO_VERSION_INFO_API in the configuration file.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.3.3 DioFlipChannelApi

Table 24 DioFlipChannelApi

Name:	DioFlipChannelApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_Cfg.h		
Range:	TRUE – Dio_FlipChannel API enabled FALSE – Dio_FlipChannel API disabled		
Default Value:	FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable the availability of the API Dio_FlipChannel() to invert the level of the Dio channel. This configuration parameter is named as DIO_FLIP_CHANNEL_API in the configuration file.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.3.4 DioPBFixedAddress

Table 25 DioPBFixedAddress

Name:	DioPBFixedAddress		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_Cfg.h		
Range:	TRUE FALSE		
Default Value:	FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This configuration parameter allows selecting the Loadable fixed address feature. TRUE – DioPBFixedAddress indicates the configuration parameter set will be loadable fixed address. The feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary		

	indirection (HisGen0010). The multiplicity of the DioConfigSet is 1...1. FALSE – Loadable fixed address feature is not used. Hence the optimization based on the requirement HisGen0010 is also not supported. The multiplicity of the DioConfigSet is 1...*. This configuration parameter is named as DIO_PB_FIXED_ADDRESS in the configuration file. The Value of the parameter DIO_PB_FIXED_ADDRESS is STD_ON / STD_OFF.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.3.5 DioUserModelInitApiEnable Enable/Disable

Table 26 Enabling/Disabling SFR resetting at Initialization

Syntax:	DioUserModelInitApiEnable		
Source:	This is IFX specific parameter		
File:	Dio_Cfg.h		
Range:	Values: TRUE: FALSE: Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable protected register access in all the Init API. If DioUserModelInitApiEnable is enabled then the Dio Init API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch DioRunningInUser0Mode.		
SFRs Modified			
Scope:	Driver.		
Dependency:	None		

5.3.6 DioUserModeRuntimeApiEnable Enable/Disable

Table 27 Enabling/Disabling SFR resetting at Initialization

Syntax:	DioUserModeRuntimeApiEnable		
Source:	This is IFX specific parameter		
File:	Dio_Cfg.h		
Range:	Values: TRUE: FALSE: Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--

	Post build	--	--
Description:	Pre-processor switch to enable / disable protected register access in all the APIs other than Init/Deinit APIs. If DioUserModeRuntimeApiEnable is enabled then all APIs other than Dio Init APIs will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch DioRunningInUser0Mode		
SFRs Modified			
Scope:	Driver.		
Dependency:	None		

5.3.7 DioRunningInUser0Mode Enable/Disable

Table 28 Enabling/Disabling SFR register access in USER0 mode

Syntax:	DioRunningInUser0Mode		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: FALSE: Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	If 'DioRunningInUser0Mode' is enabled then the API's selected by 'DioUserModeInitApiEnable' or 'DioUserModeRuntimeApiEnable' will run in user mode 0. If the 'DioRunningInUser0Mode' switch is not selected and if any of the switches 'DioUserModeInitApiEnable', 'DioUserModeRuntimeApiEnable' are selected then those APIs will run in user mode 1. This switch selects between user mode0 and user mode 1 DioRunningInUser0Mode – STD_ON implies User Mode 0 DioRunningInUser0Mode – STD_OFF implies User Mode 1 if any of the switches (DioUserModeInitApiEnable or DioUserModeRuntimeApiEnable) are true else Supervisor Mode (If all the switches mentioned are OFF).		
SFRs Modified			
Scope:	Driver.		
Dependency:	If 'DioRunningInUser0Mode' is enabled then at least any one of the switches 'DioUserModeInitApiEnable' or 'DioUserModeRuntimeApiEnable' should be enabled.		

5.4 Container: DioConfig

This container is the base of multiple configuration sets. It contains lists/containers for configuration of Dio Ports, Dio Channels and Dio Channel Groups.

Its multiplicity is 1 or 1...*, if DioPBFixedAddress is enabled or disabled respectively.

5.4.1 Container: DioPort

Configuration of user defined DIO port.

This is a sub container under the container DioConfig for configuration of an individual DIO channel.

Its multiplicity is 1...(number of Ports on the Microcontroller).

Note: Besides a HW specific DIO port name which is typically fixed for a specific micro controller, additional symbolic names can be defined per DIO port. This container definition does not explicitly define a symbolic name parameter. Instead, the container's name will be used in the ECU Configuration Description to specify the symbolic name of the DIO port. This symbolic name is #defined in the code.

Hence the container's should not clash with other symbolic names and published symbolic names. Also they must comply with C standard.

In a single configuration set, the container's name should be unique. However, in a multiple configuration sets, same container name can be reused across the config sets(not applicable for TC27XBC release).

Its multiplicity is 1...(number of Pins on the Microcontroller for specified port).

5.4.1.1 DioPortId

Table 29 Dio Port Id

Name:	DioPortId		
Source:	This parameter is defined by the AUTOSAR.		
File:	Symbolic names are generated in Dio_Cfg.h; however the value depends on the Post build configuration.		
Range:	The value is hardware dependent		
Default value:	ID corresponding to the port		
Configuration Class:	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	This parameter is the numeric identifier of the DIO port. Notes: - This numeric value should correspond to the port number. E.g.: Port0 should be numbered 0, Port1 as 1... and Port40 as 40. - This value will be assigned to the DIO port symbolic name (i.e. the name of the container DioPort). - Not all MCU ports may be used for DIO, thus there may be "gaps" in the list of all IDs.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.4.2 Container: DioChannel

This is a sub container under the container DioPort for configuration of an individual DIO channel.

Note: Besides a HW specific channel name which is typically fixed for a specific micro controller, additional symbolic names can be defined per channel. This container definition does not explicitly define a symbolic name parameter. Instead, the container's name will be used in the ECU Configuration Description to specify the symbolic name of the channel. This symbolic name is #defined in the code.

Hence the container's name should not clash with other symbolic names and published symbolic names. Also they must comply with C standard.

In a single configuration set, the container's name should be unique. However, in a multiple configuration sets, same container name can be reused across the config sets(not applicable for TC27XBC release).

Its multiplicity is 0...(number of Pins on the Microcontroller for specified port).

5.4.2.1 DioChannelId

Table 30 Dio Channel Id

Name:	DioChannelId		
Source:	This parameter is defined by the AUTOSAR.		
File:	Symbolic names are generated in Dio_Cfg.h; however the value depends on the Post build configuration.		
Range:	The value depends on the available pins in a port		
Default value:	ID corresponding to the port pin		
Configuration Class:	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	Numeric identifier of the DIO channel. This value is assigned to the symbolic name of the DIO channel.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.4.3 Container: DioChannelGroup

The DioChannelGroup container holds configuration parameters that apply to each channel group. This is a sub container under DioPort container. Its multiplicity is 0...*.

A channel group represents several adjoining DIO channels represented by a logical group. There can be as many groups as required by the user. The Mask value (a value with adjoining one's) should be defined by the user to get the DioChannelGroup parameters of type Dio_ChannelGroupType. The DioPortOffset value is calculated from the DioPortMask.

Its multiplicity is 0...(number of Pins on the Microcontroller for specified port).

5.4.3.1 DioChannelGroupIdentification

Table 31 Dio Channel Group Identification

Name:	DioChannelGroupIdentification		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_Cfg.h		
Range:	User defined C compatible macro name		
Default value:	DioChGrpId_Px_y Where x is Port number, y is group number		
Configuration Class:	Pre-compile	x	VariantPC
	Link Time	--	--
	Post build	--	--

Description:	User defined symbolic name of this DIO channel group. User defined Symbolic Names are #defined in the code. So they must be unique and should not clash with other symbolic names and published symbolic names. Also they must comply with C standard. Note: In a single configuration set, the parameter name should be unique. However, in a multiple configuration sets, same container name can be reused across the config sets(not applicable for TC27XBC release).
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.4.3.2 DioPortMask

Table 32 Dio Port Mask

Name:	DioPortMask		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_PBCfg.c		
Range:	0x1 to 0xFFFF		
Default value:	0		
Configuration Class:	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	This shall be the mask which defines the positions of the channel group. The data type depends on the port width. The user must ensure to enter a value with adjoining ones. For instance 0x101 is not valid, while 0x0110 is valid.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.4.3.3 DioPortOffset

Table 33 Dio Port Offset

Name:	DioPortOffset		
Source:	This parameter is defined by the AUTOSAR.		
File:	Dio_PBCfg.c		
Range:	0 to width of port (Port width is microcontroller dependent)		
Default value:	0		
Configuration Class:	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB

Description:	DioPortOffset is the starting position of the Channel Group on the port, counted from the LSB. The user need not configure this parameter, since the code generator calculates its value depending on the value configured in DioPortMask.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.5 Container: DioSafety

This container contains the configuration parameters for the safety features of the DIO driver. It is 1:1.

5.5.1 DioSafetyEnable

Table 34 DioSafetyEnable

Name:	DioSafetyEnable		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	TRUE - Safety features enabled for DIO driver FALSE – Safety feature disabled		
Default value:	FALSE		
Configuration Class:	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety features of DIO driver. By enabling this flag, implicitly the following safety features are enabled: - Range check on API parameters shall be done - DIO configuration structure shall have a unique marker field which will be verified by the driver(applicable for AS403) - Publish the safety usage of DIO driver for user to set the respective MPU settings		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

5.5.2 DioInitCheckApi

Table 35 DioInitCheckApi

Name:	DioInitCheckApi		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	TRUE - Dio_InitCheck() API will be available FALSE - Dio_InitCheck() API will be not available		
Default value:	FALSE		
Configuration Class:	Pre-compile	X	All Variants
	Link Time	--	--

	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Dio_InitCheck () API which verifies the initialization done by DIO driver via Dio_Init ().		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	DioSafetyEnable should be enabled for this parameter to be available.		

5.6 Container: CommonPublishedInformation

This container contains the common published information. These parameters are non-editable.

5.6.1 ArMajorVersion

Table 36 ArMajorVersion

Name:	ArMajorVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h
Range:	0 - 255
Default value:	4
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.2 ArMinorVersion

Table 37 ArMinorVersion

Name:	ArMinorVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h
Range:	0 - 255
Default value:	0
Description:	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.3 ArPatchVersion

Table 38 ArPatchVersion

Name:	ArPatchVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h

Name:	ArPatchVersion
Range:	0 - 255
Default value:	3
Description:	Patch version number of AUTOSAR specification on which the appropriate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.4 SwMajorVersion

Table 39 SwMajorVersion

Name:	SwMajorVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h
Range:	0 - 255
Default value:	Refer Dio_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.5 SwMinorVersion

Table 40 SwMinorVersion

Name:	SwMinorVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h
Range:	0 - 255
Default value:	Refer Dio_Cfg.h
Description:	Minor version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.6 SwPatchVersion

Table 41 SwPatchVersion

Name:	SwPatchVersion
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h
Range:	0 - 255

Name:	SwPatchVersion
Default value:	Refer Dio_Cfg.h
Description:	Patch version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.7 ModuleId

Table 42 ModuleId

Name:	ModuleId
Source:	This parameter is defined by IFX
File:	Dio.h
Default value:	120
Description:	DIO Driver module ID as described by AUTOSAR.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.8 VendorId

Table 43 VendorId

Name:	VendorId
Source:	This parameter is defined by IFX
File:	Dio.h
Default value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list. Vendor is Infineon Technologies.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.6.9 VendorApilnfix

Table 44 VendorApilnfix

Name:	VendorApilnfix
Source:	This parameter is defined by IFX
File:	Dio.xdm
Default value:	None
Description:	Vendor Specific Name used for extending the API names along with VendorId.
SFRs Modified:	None
Scope:	Not Applicable for DIO driver

Name:	VendorApilInfix
Dependency:	None

5.6.10 Release

Table 45 Release

Name:	Release
Source:	This parameter is defined by IFX
File:	Dio.xdm/.bmd
Range:	_TRICORE_TC2XX, X, varies as per the derivate Eg: _TRICORE_TC275, _TRICORE_TC277, for TC27x,
Description:	Aurix derivative used for implementation.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

5.7 Derived Configuration Parameters

This section describes about the configuration parameters whose values are derived from the values of other parameters configured in the tool. These configuration parameters are available only in the configuration files and not in the tool.

5.7.1 Dio_kChannelGroupConfig

Table 46 Dio_kChannelGroupConfig

Name:	Dio_kChannelGroupConfig		
Source:	This parameter is defined by IFX		
File:	Dio_PBCfg.c		
Range:	Refer to the description on the type definition Dio_ChannelGroupType in the DIO specification [2]		
Configuration Class	Pre-Compile	--	
	Link Time	--	
	Post Build	x	Variant PB
Description:	This parameter contains the configuration of the DIO channel groups		
Scope:	Module		
Dependency:	This structure is dependant on the configuration of the DIO channel groups for all the ports available in the microcontroller.		

5.7.2 DIO_PORTS_AVAILABLE_00_31

Table 47 DIO_PORTS_AVAILABLE_00_31

Name:	DIO_PORTS_AVAILABLE_00_31
Source:	This parameter is defined by IFX
File:	Dio_Cfg.h

Range:	Bit value = 0 implies the port is not available Bit value = 1 implies the port is available Bit 0 is for Port 0, Bit 1 is for Port 1, ... , Bit 31 is for Port 31		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	This parameter specifies the ports that are available on the microcontroller		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.3 DIO_PORTS_AVAILABLE_32_40

Table 48 DIO_PORTS_AVAILABLE_32_40

Name:	DIO_PORTS_AVAILABLE_32_40		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	Bit value = 0 implies the port is not available Bit value = 1 implies the port is available Bit 0 is for Port 32, Bit 1 is for Port 33, ... , Bit 8 is for Port 40		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	This parameter specifies the ports that are available on the microcontroller		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.4 DIO_PORTS_READONLY_00_31

Table 49 DIO_PORTS_READONLY_00_31

Name:	DIO_PORTS_READONLY_00_31		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	Bit value = 0 implies the port readable/writable Bit value = 1 implies the port is read only port Bit 0 is for Port 0, Bit 1 is for Port 1, ... , Bit 31 is for Port 31		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	Specifies the ports that are read only ports on the microcontroller		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.5 DIO_PORTS_READONLY_32_40

Table 50 DIO_PORTS_READONLY_32_40

Name:	DIO_PORTS_READONLY_32_40		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	Bit value = 0 implies the port readable/writable Bit value = 1 implies the port is read only port Bit 0 is for Port32, Bit 1 is for Port 33, ... , Bit 8 is for Port 40		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	Specifies the ports that are read only ports on the microcontroller		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.6 DIO_PORT_<x>_MSPIN

Table 51 DIO_PORT_<x>_MSPIN

Name:	DIO_PORT_<x>_MSPIN, x = 0 – 41		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	0 – 15, dependent on the microcontroller		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	These parameters specify the values of the most significant pin on the port Note: In case the port is not available on the hardware, the value is kept 0		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.7 DIO_MASK_UNDEF_PINS_PORT<x>

Table 52 DIO_MASK_UNDEF_PINS_PORT<x>

Name:	DIO_MASK_UNDEF_PINS_PORT<x>, x = 0 – 41		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	0x0000 – 0xFFFF, dependent on the hardware		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	The parameter specifies the value to mask the undefined port pins within a port		
Scope:	Module		

Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.
--------------------	---

5.7.8 DIO_CHANNEL_<Port>_<Pin>

Table 53 DIO_CHANNEL_<Port>_<Pin>

Name:	DIO_CHANNEL_<Port>_<Pin>; Port = 0 – 41, Pin = 0 – 15		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	0x00 – 0xFFFF; the value is hardware dependent		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	This parameter specifies the symbolic name for the DIO channels		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.9 DIO_PORT_<x>

Table 54 DIO_PORT_<x>

Name:	DIO_PORT_<x>, x = 0 – 41		
Source:	This parameter is defined by IFX		
File:	Dio_Cfg.h		
Range:	0 – 40; the value is hardware dependent		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	
	Post Build	--	
Description:	This parameter specifies the symbolic name for the DIO ports		
Scope:	Module		
Dependency:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.		

5.7.10 Dio_PortIdConfigured

Table 55 Dio_PortIdConfigured

Name:	Dio_PortIdConfigured		
Source:	This parameter is defined by IFX		
File:	Dio_PBCfg.c		
Range:	UInt16		
Configuration Class	Pre-Compile	--	
	Link Time	--	
	Post Build	x	VariantPB
Description:	The Value of the parameter Dio_PortIdConfigured is based on the configured Dio channels.		

	Dio Channel 0 of the port is assigned to bit 0, Dio Channel 1 of the port is assigned to bit 1, . . . Dio Channel 15 of the port is assigned to bit 15 If the Dio channel is configured, respective bits of the pins gets the value 1. The derived parameter Dio_PortIdConfigured is the member of structure Dio_PortChannelIdType.
Scope:	Module
Dependency:	None

5.7.11 Dio_ChannelGrpConfigSize

Table 56 Dio_ChannelGrpConfigSize

Name:	Dio_ChannelGrpConfigSize		
Source:	This parameter is defined by IFX		
File:	Dio_PBCfg.c		
Range:	UInt32		
Configuration Class	Pre-Compile	--	
	Link Time	--	
	Post Build	x	VariantPB
Description:	The Value of the parameter Dio_ChannelGrpConfigSize defines the number of Dio channel groups are configured. The derived parameter Dio_ChannelGrpConfigSize is the member of structure Dio_ConfigType. Numbers of channel groups configured are defined as DIO_CHANNELGROUPCOUNT_x. where x is the no of the DioConfigSet.		
Scope:	Module		
Dependency:	Number of Channel groups configured by the user		

6 Published parameters

This section describes the parameters published by the DIO driver.

6.1 VendorId

Table 57 VendorId

Syntax:	DIO_VENDOR_ID
Type:	Macro
File:	Dio.h
Value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list

6.2 ModuleId

Table 58 ModuleId

Syntax:	DIO_MODULE_ID
Type:	Macro
File:	Dio.h
Value:	0x78
Description:	Module ID of this Driver from Module List as described by AUTOSAR

6.3 InstanceId

Table 59 Module instance id

Syntax:	DIO_INSTANCE_ID
Type:	Macro
File:	Dio_Cfg.h
Value:	0x0
Description:	Instance ID of this driver from Module List as described by AUTOSAR

6.4 ARReleaseMajorVersion

Table 60 Dio Driver AUTOSAR Major Version

Syntax:	DIO_AR_RELEASE_MAJOR_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.

6.5 ARReleaseMinorVersion

Table 61 DIO Driver AUTOSAR Minor Version

Syntax:	DIO_AR_RELEASE_MAJOR_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.

6.6 ARReleaseRevisionVersion

Table 62 Dio Driver AUTOSAR Patch Version

Syntax:	DIO_AR_RELEASE_REVISION_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

6.7 SWMajorVersion

Table 63 Dio Driver SW Implementation Major Version

Syntax:	DIO_SW_MAJOR_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The Major version number is incremented if the driver is not downward compatible any more (e.g. existing API changed)

6.8 SWMinorVersion

Table 64 Dio Driver SW Implementation Minor Version

Syntax:	DIO_SW_MINOR_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Minor version number of the vendor specific implementation of the Driver. The numbering is vendor specific. Minor version number is incremented if the driver is still downward compatible (e.g. new functionality added)

6.9 SWPatchVersion

Table 65 Dio SW Implementation Patch Version

Syntax:	DIO_SW_PATCH_VERSION
Type:	Macro
File:	Dio_Cfg.h
Value:	Refer Dio_Cfg.h
Description:	Patch version number of the vendor specific implementation of the driver. The numbering is vendor specific. The patch version number is incremented if the driver is still upwards and downwards compatible (e.g. bug fixed)

7 API Documentation

7.1 API Type Definitions

7.1.1 Type Definition Dio_ChannelType

Table 66 Type Definition: Dio_ChannelType

Syntax:	Dio_ChannelType
Type:	Uint16
File:	Dio.h
Range:	All defined IO port pins
Description:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.

7.1.2 Type Definition Dio_PortType

Table 67 Type Definition: Dio_PortType

Syntax:	Dio_PortType
Type:	uint8
File:	Dio.h
Range:	All defined IO ports
Description:	The value of this parameter is hardware dependent and depends on the microcontroller derivative used.

7.1.3 Type Definition Dio_ChannelGroupType

Table 68 Type Definition: Dio_ChannelGroupType

Syntax:	Dio_ChannelGroupType	
Type:	struct	
File:	Dio.h	
Range:	Dio_PortLevelType mask	Defines the pins that forms the channel group
	uint8 offset	Defines the position of the channel group on the "port", counted from LSB.
	Dio_PortType port	The port on which channel group is defined
Description:	Several adjoining DIO channels within a DIO port can be combined to a formal logical channel group. This type definition is used to configure and refer the channel groups. Channel groups defined by user are used by two DIO driver services namely Dio_ReadChannelGroup and Dio_WriteChannelGroup.	

7.1.4 Type Definition Dio_LevelType

Table 69 Type Definition: Dio_LevelType

Syntax:	Dio_LevelType
Type:	uint8
File:	Dio.h
Range:	STD_HIGH – This has value 1 and represent physical state 5V or 3.3V STD_LOW – This has value 0 and represent physical state 0V
Description:	The macros STD_HIGH and STD_LOW are imported from file Std_Types.h

7.1.5 Type Definition Dio_PortLevelType

Table 70 Type Definition: Dio_PortLevelType

Syntax:	Dio_PortLevelType
Type:	uint16
File:	Dio.h
Range:	0 to 0xFFFF.
Description:	Dio_PortLevelType inherits the size of the largest port. Each bit in this type represents level of one pin of the port. For example bit 0 represents level of pin 0 in the port.

7.1.6 Type definition Dio_ConfigType

Table 71 Type definition Dio_ConfigType

Syntax:	Dio_ConfigType	
Type:	Struct	
File:	Dio.h	
Range	UInt32 Dio_MarkerCheckValue	Holds the module id and instance id information; required only when DIO_SAFETY_ENABLE is STD_ON
	const Dio_PortChannelIdType* Dio_PortIdConfigPtr	It holds the information for the Portpins configured. Refer 5.7.10
	const Dio_ChannelGroupType* Dio_ChannelGroupConfigPtr	Refer 5.7.1
	uint32 Dio_ChannelGroupConfigSize	It is the number of channel group configured for the configuration. Refer 5.7.11
	Dio_ChannelGroupType Dio_ChGrpIdMap	It is the DioChannelGroup identifier map structure which holds the mapping of channel Group Ids. Applicable only in a multi-config set (not applicable for TC27XBC release).

	uint16 Dio_PortPinIdMap	It is the DioChannel identifier map array which holds the mapping of PortPin Ids. Applicable only in a multi-config set (not applicable for TC27XBC release).
	uint8 Dio_PortIdMap	It is the DioPort identifier map array which holds the mapping of Port Ids. Applicable only in a multi-config set (not applicable for TC27XBC release).
Description:	This type definition is applicable to the configuration data used in the API Dio_Init. This is type for all post build configurable parameters. Dio_PortChannelIdConfig: - Gives the configuration values for the configured Dio ports and channels. Dio_ChannelGroupConfig: - Gives the configuration values for the configured channel groups. NULL_PTR is used if channel groups are not configured. Dio_ChannelGrpConfigSize: - Gives number of channel groups configured. Dio_PortIdMap: - It is the DioPort identifier map array which holds the mapping of Port Ids Dio_PortPinIdMap: - It is the DioChannel identifier map array which holds the mapping of PortPin Ids. Dio_ChGrpIdMap: It is the DioChannelGroup identifier map structure which holds the mapping of channel Group Ids.	

7.2 API Functions

This chapter describes the API interfaces provided by Dio Driver.

7.2.1 Dio_Init

Table 72 Specification of API Dio_Init

Service name:	Dio_Init
Syntax:	<pre>void Dio_Init (const Dio_ConfigType* ConfigPtr)</pre>
Service ID:	0x10
Sync/ Async:	Synchronous
Origin:	Autosar
Re-entrancy:	Non Re-entrant

Parameters (in):	ConfigPtr	Pointer to post build configuration data
Parameters (out):	None	
Return value:	None	
Description:	The Initialization service initializes the Dio current configuration. Resetting the SFRs, used by DIO driver is done by Port_Init() API in Port driver when PortResetSfrAtInit is enabled.	
Caveats:	--	
Configuration:	--	
DET:	DIO_E_PARAM_CONFIG is reported when passed ConfigPtr parameter is NULL pointer in VariantPB mode.	
Safety Error Reporting:	When the passed ConfigPtr is NULL or if marker check fails, DIO_E_PARAM_CONFIG is reported.	
DEM:	No DEM for DIO Driver.	
Implementation Comments:	Initialization of global variables. The API is expected to call before use of any other Dio API call.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModelInitApiEnable is 'False'.	

7.2.2 Dio_ReadChannel

Table 73 Specification of API Dio_ReadChannel

Service name:	Dio_ReadChannel	
Syntax:	Dio_LevelType Dio_ReadChannel (Dio_ChannelType ChannelId) 	
Service ID:	0x00	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	ChannelId	ID of DIO channel
Parameters (out):	None	
Return value:	STD_HIGH	The physical level of the corresponding Pin is STD_HIGH
	STD_LOW	The physical level of the corresponding Pin is STD_LOW
Description:	- The level of a single DIO channel is read with this service - This service returns the level of the specified DIO channel, i.e., STD_HIGH or STD_LOW. If development errors are enabled and an error occurred, this service will return with the value '0' - This service works on input and output channels. To read the value of the output channel, this service reads the pin level contained in Pn_IN register, i.e., direct	

	read back of the pin.
Caveats:	--
Configuration:	Symbolic names for each configured DIO channel is provided by the configuration process.
DET:	DET error DIO_E_PARAM_INVALID_CHANNEL_ID is reported if the parameter ChannelId is invalid i.e., The ID passed is of : • Channels > Max. Supported channels. • Channels of ports > Max port. • Channels of ports that do not exist. • Channels not configured for the current configuration DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the service returns the value STD_LOW to the calling function.
Safety Error Reporting:	DIO_E_PARAM-INVALID_CHANNEL_ID is reported when incorrect channelId is passed to this API.
DEM:	No DEM for DIO Driver.
Implementation Comments:	Aurix microcontrollers support the direct read-back of a pin value from Port Input register even if Port line is configured as output. Hence, the DIO read services shall provide the real pin level, even when they are used on a channel which is configured as an output channel.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.

7.2.3 Dio_WriteChannel

Table 74 Specification of API Dio_WriteChannel

Service name:	Dio_WriteChannel	
Syntax:	<pre>void Dio_WriteChannel (Dio_ChannelType ChannelId, Dio_LevelType Level) </pre>	
Service ID:	0x01	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	ChannelId	ID of DIO Channel
	Level	Desired output Level of the Channel,
Parameters (out):	None	
Return value:	None	
Description:	• This service is to modify the levels of output channels • The level of a single DIO channel shall be set by this service to STD_HIGH or	

	STD_LOW • This service works on input and output channels. • If the specified channel is configured as an output channel, the Dio_WriteChannel service will set the specified Level for the specified channel. • If the specified channel is configured as an input channel, the Dio_WriteChannel service will write the level to the output latch, but will have no influence on the physical level of the channel. • If the specified channel is configured as an input channel, the Dio_WriteChannel service will have no influence on the result of the next Read-Service. • If development errors are enabled and an error occurred, the functionality of the API is not performed.
Caveats:	--
Configuration:	Symbolic names for each configured DIO channel is provided by the configuration process.
DET:	DET error DIO_E_PARAM_INVALID_CHANNEL_ID is reported if the parameter ChannelId is invalid i.e., The ID passed is of : • Channels > Max. Supported channels. • Channels of ports > Max port. • Channels of ports that do not exist. • Channels not configured for the current configuration • Channels of read only ports. DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the write operation is not performed.
Safety Error Reporting	DIO_E_PARAM_INVALID_CHANNEL_ID is reported when incorrect ChannelID is passed to this API. DIO_E_PARAM_INVALID_LEVEL is reported when incorrect value for Level is passed.
DEM:	No DEM for DIO driver.
Implementation Comments:	The level specified in the parameter Level is written to the output latch of the channel irrespective of the direction configured for the channel.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.

7.2.4 Dio_FlipChannel

Table 75 Specification of API Dio_FlipChannel

Service name:	Dio_FlipChannel
Syntax:	<pre>Dio_LevelType Dio_FlipChannel (Dio_ChannelType ChannelId)</pre>
Service ID:	0x11
Sync/ Async:	Synchronous

Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	ChannelId	ID of DIO channel
Parameters (out):	None	
Return value:	STD_HIGH	The physical level of the corresponding Pin is STD_HIGH
	STD_LOW	The physical level of the corresponding Pin is STD_LOW
Description:	- This service is to flip the levels of output channels - The level of a single DIO channel shall be toggled by this service to STD_HIGH or STD_LOW - This service works on input and output channels. - If the specified channel is configured as an output channel, the Dio_FlipChannel service will flip the Level of the specified channel and return the flipped Level. - If the specified channel is configured as an input channel, the Dio_FlipChannel service will write the flipped level to the output latch, but will have no influence on the physical level of the channel and return the actual level of the input channel. - If the specified channel is configured as an input channel, the Dio_FlipChannel service will have no influence on the result of the next Read-Service. - If development errors are enabled and an error occurred, the functionality of the API is not performed.	
Caveats:	--	
Configuration:	This API is available only if DioFlipChannelApi is enabled. Symbolic names for each configured DIO channel is provided by the configuration process.	
DET:	DET error DIO_E_PARAM_INVALID_CHANNEL_ID is reported if the parameter ChannelId is invalid i.e., The ID passed is of : - Channels > Max. Supported channels. - Channels of ports > Max port. - Channels of ports that do not exist. - Channels not configured for the current configuration - Channels of read only ports. DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the write operation is not performed and returns STD_LOW to the calling function.	
Safety Error Reporting:	DIO_E_PARAM_INVALID_CHANNEL_ID is reported when incorrect channelId is passed to this API.	
DEM:	No DEM for DIO Driver.	
Implementation Comments:	The flipped level of the specified channel is written to the output latch of the channel irrespective of the direction configured for the channel.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.	

7.2.5 Dio_ReadPort

Table 76 Specification of API Dio_ReadPort

Service name:	Dio_ReadPort	
Syntax:	Dio_PortLevelType Dio_ReadPort (Dio_PortType PortId)	
Service ID:	0x02	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	PortId	ID of DIO Port
Parameters (out):	None	
Return value:	Dio_PortLevelType	The return value depends on whether read back is supported or not.
Description:	- This service reads the levels of all channels of one port and returns this value to the calling function. A bit value '0' indicates the corresponding channel to physical STD_LOW and a bit value '1' indicates the corresponding channel to physical STD_HIGH. - When reading a port which is smaller than the Dio_PortType, the bits corresponding to undefined port pins are 0. - This service works on both input and output channels - For reading the level of the output pins of a port, this service reads the pin level contained in Pn_IN register, i.e., direct read back of the pin is done.	
Caveats:	--	
Configuration:	Symbolic names for each configured DIO Port are provided by the configuration process.	
DET:	DET error DIO_E_PARAM_INVALID_PORT_ID is reported if the parameter PortId is invalid i.e., The ID passed is of : - Ports > Max port. - Ports that do not exist. - Ports not configured for the current configuration DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the service is returned with the value 0	
Safety Error Reporting:	DIO_E_PARAM_INVALID_PORT_ID is reported when incorrect PortID is passed to this API.	
DEM:	No DEM for DIO driver.	
Implementation Comments:	Aurix microcontrollers support direct read-back of a pin value from Port Input register even if Port line is configured as output. Hence, the service provides the real pin level, when they are used on a channel which is configured as an output channel.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.	

7.2.6 Dio_WritePort

Table 77 Specification of API Dio_WritePort

Service name:	Dio_WritePort	
Syntax:	<pre>void Dio_WritePort (Dio_PortType PortId, Dio_PortLevelType Level)</pre>	
Service ID:	0x03	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	PortId	ID of DIO Port
	Level	Output Levels of the channels of the port to be set
Parameters (out):	None	
Return value:	None	
Description:	- The service sets the specified level to the specified port. - All the port pins are written simultaneously. Value '0' corresponds to STD_LOW and '1' corresponds to STD_HIGH - When writing a port which is smaller than the Dio_PortType the MSB is ignored - This service works on both input and output channels - The physical levels of the DIO Channels that are configured as input remain unchanged.	
Caveats:	--	
Configuration:	Symbolic names for each configured DIO Port are provided by the configuration process.	
DET:	DET error DIO_E_PARAM_INVALID_PORT_ID is reported if the parameter PortId is invalid i.e., The ID passed is of : - Ports > Max port. - Ports that do not exist. - Ports are read only ports. - Ports not configured for the current configuration DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the write operation is not performed.	
Safety Error Reporting:	DIO_E_PARAM_INVALID_PORT_ID is reported when incorrect PortID is passed to this API.	
DEM:	No DEM for DIO driver.	
Implementation Comments:	The level specified in the parameter Level is written to the output latch of the port irrespective of the direction configured for the channels.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.	

7.2.7 Dio_ReadChannelGroup

Table 78 Specification of API Dio_ReadChannelGroup

Service name:	Dio_ReadChannelGroup	
Syntax:	<pre>Dio_PortLevelType Dio_ReadChannelGroup (const Dio_ChannelGroupType *ChannelGroupPtr)</pre>	
Service ID:	0x04	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	ChannelGroupPtr	Pointer to ChannelGroup
Parameters (out):	None	
Return value:	Dio_PortLevelType	The return value depends on whether read back is supported or not.
Description:	- This API masks the channel group and reads the levels of its channels. A bit value '0' indicates the corresponding pin to physical STD_LOW, a bit value '1' indicates the corresponding channel to physical STD_HIGH. The values read by this API are aligned to the LSB - This service works on both input and output channels of the channel group.	
Caveats:	--	
Configuration:	--	
DET:	The error DIO_E_PARAM_INVALID_GROUP is reported if the channel group configured is invalid i.e., The channel group passed contains any of the following configurations : - ChannelGroupPtr passed is a Null - Ports > Max port - Ports that do not exist Mask value is 0 - Mask value is greater than the port width - Offset value is greater than the port width - Channel group not configured for the current configuration DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the service returns the value 0 to the calling function	
Safety Error Reporting:	DIO_E_PARAM_INVALID_GROUP is reported when incorrect ChannelGroupID is passed to this API.	
DEM:	No DEM for DIO driver.	
Implementation Comments:	Aurix microcontrollers support the direct read-back of a pin value from Port Input register even if Port line is configured as output. Hence, the service shall provide the real pin level, when they are used on a channel which is configured as an output channel.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.	

7.2.8 Dio_WriteChannelGroup

Table 79 Specification of API Dio_WriteChannelGroup

Service name:	Dio_WriteChannelGroup	
Syntax:	<pre>void Dio_WriteChannelGroup (const Dio_ChannelGroupType *ChannelGroupIDPtr, Dio_PortLevelType Level)</pre>	
Service ID:	0x05	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	ChannelGroupIDPtr	Pointer to ChannelGroup
	Level	Desired Output Level of the channels of the ChannelGroup
Parameters (out):	None	
Return value:	None	
Description:	• This API does the masking and writes simultaneously to the channels within the group. A bit value '0' corresponds to STD_LOW and '1' to STD_HIGH. This API will shift the value depending on the offset value and then writes to the channel group. • Read-modify-write sequences are not interruptible in this service. • This service works on both input and output channels configured in the group. • The physical level of the input channels within the channel group is not changed by the write operation of this API.	
Caveats:	--	
Configuration:	--	
DET:	The error DIO_E_PARAM_INVALID_GROUP is reported if the channel group configured is invalid i.e., The channel group passed contains any of the following configurations : • GroupIDPtr passed is a Null • Ports > Max port • Ports that do not exist Mask value is 0 • Read only ports. • Mask value is greater than the port width • Offset value is greater than the port width • Channel group not configured for the current configuration DET error DIO_E_UNINIT is reported, if this API is called prior to Dio_Init. In this case the service returns the value 0 to the calling function.	
Safety Error Reporting:	DIO_E_PARAM_INVALID_GROUP is reported when incorrect ChannelGroupID is passed to this API.	
DEM:	No DEM for DIO driver.	
Implementation Comments:	The level specified in the parameter Level is written to the output latch of the port irrespective of the direction configured for the channels.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'.	

This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.

7.2.9 Dio_GetVersionInfo

Table 80 Specification of API Dio_GetVersionInfo

Service name:	Dio_GetVersionInfo	
Syntax:	<pre>void Dio_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)</pre>	
Service ID:	0x12	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	None	
Parameters (out):	VersionInfoPtr	Pointer to where to store the version information of this driver.
Return value:	None	
Description:	This service returns the version information of this module. The version information includes: - Module Id - Vendor Id - Vendor specific version numbers.	
Caveats:	--	
Configuration:	This service is only available if enabled by the pre-processor switch DIO_VERSION_INFO_API.	
DET:	The error DIO_E_PARAM_POINTER is reported if the • VersionInfoPtr passed is a Null In this case the service returns the value 0 to the calling function.	
Safety Error Reporting:	NA,	
DEM:	No DEM for DIO driver.	
Implementation Comments:	This API is available only if the switch DIO_VERSION_INFO_API is enabled by the user.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModeRuntimeApiEnable is 'False'.	

7.2.10 Dio_InitCheck

Table 81 Specification of API Dio_InitCheck

Service name:	Dio_InitCheck	
Syntax:	Std_ReturnType Dio_InitCheck (const Dio_ConfigType *ConfigPtr)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin:	Vendor specific	
Re-entrancy:	Non Re-entrant	
Parameters (in):	ConfigPtr	Pointer to post build configuration data
Parameters (out):	None	
Return value:	E_OK – Initialization comparison successful E_NOT_OK – Initialization not properly done by DIO driver.	
Description:	This is an IFX API, which should be called after Dio_Init to verify the initialization done by DIO driver against the configuration pointer only.	
Caveats:	--	
Configuration:	This API is available only if DioInitCheckApi is enabled.	
DET:	None	
Safety Error Reporting:	No Error.	
DEM:	No DEM for DIO Driver.	
Implementation Comments:	Verifies the initialization of global variables by Dio_Init The API is expected to be called immediately after Dio_Init API.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter DioRunningInUser0Mode is 'True' and DioUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter DioRunningInUser0Mode is 'False' and DioUserModelInitApiEnable is 'False'.	

7.3 Interrupt Handling

No interrupts are handled by DIO driver.

8 Error classification

Table 82 Error Classification for DETs

Error Code	Error	Relevance	Value (hex)
DIO_E_PARAM_INVALID_CHANNEL_ID	Invalid channel name requested	Development	0x0A
DIO_E_PARAM_INVALID_PORT_ID	Invalid port name requested	Development	0x14
DIO_E_PARAM_INVALID_GROUP	Invalid ChannelGroup requested	Development	0x1F
DIO_E_PARAM_CONFIG	Invalid ConfigPtr is passed (e.g. NULL Pointer)	Development	0x10
DIO_E_PARAM_POINTER	Dio_GetVersionInfo called with NULL_PTR(it is an IFX DET)	Development	0x20
DIO_E_UNINIT	Dio module not initialized(it is an IFX DET)	Development	0xAA

Table 83 Error Classification for Safety

Type of error	Related error code	Value[hex]
Invalid channel name requested	DIO_E_PARAM_INVALID_CHANNEL_ID	0x0A
Invalid port name requested	DIO_E_PARAM_INVALID_PORT_ID	0x14
Invalid ChannelGroup requested	DIO_E_PARAM_INVALID_GROUP	0x1F
Dio_WriteChannel Api called with wrong Level	DIO_E_PARAM_INVALID_LEVEL	0x21
Dio_Init Called with wrong Configuration.	DIO_E_PARAM_CONFIG	0x10

9 Example Usage

This chapter describes how to use different API provided by DIO driver.

Since the DIO driver works on the ports and port pins that are configured by PORT driver, configuration and initialization of the PORT driver is required before accessing DIO driver, Refer [\[6\]](#).

9.1 Configuration of Driver

9.1.1 DIO Driver Published Symbolic Names

MCU specific DIO channel and DIO port symbolic name are defined in the Dio_Cfg.h (derivative or board specific header file)

Code Listing 10 lists the MCU specific published symbolic names for DIO Channel of Port 0 pins and DIO Port

Code Listing 10 MCU specific DIO Channel and DIO Port published symbolic names

```

026:  /*
027:     Published symbolic names for DIO channels
028:  */
029:  /* Port 0 pins */
030:  #define DIO_CHANNEL_0_0      ((Dio_ChannelType)0x0000)
031:  #define DIO_CHANNEL_0_1      ((Dio_ChannelType)0x0001)
032:  #define DIO_CHANNEL_0_2      ((Dio_ChannelType)0x0002)
033:  #define DIO_CHANNEL_0_3      ((Dio_ChannelType)0x0003)
034:  #define DIO_CHANNEL_0_4      ((Dio_ChannelType)0x0004)
035:  #define DIO_CHANNEL_0_5      ((Dio_ChannelType)0x0005)
036:  ...
037:  ...
038:  ...
039:
040:  /*
041:     Published symbolic names for DIO ports
042:  */
043:  #define DIO_PORT_0           ((Dio_PortType)0)
044:  #define DIO_PORT_1           ((Dio_PortType)1)
045:  #define DIO_PORT_2           ((Dio_PortType)2)
046:  #define DIO_PORT_3           ((Dio_PortType)3)
047:  #define DIO_PORT_4           ((Dio_PortType)4)
048:  #define DIO_PORT_5           ((Dio_PortType)5)
049:  ...
050:  ...

```

9.1.2 Generation of DIO Configuration Data

Here an example is taken to generate the Dio configuration data for Motor pins and CAN transceiver pins. Motor pin from Port 0 Pin5 to Port 0 Pin8 channels form MOTOR_CTL_GRP Dio channel group and CAN transceiver pins Port 1 Pin 1 and Port 1 Pin 2 channels form CAN_TRCV_GRP Dio channel group.

Default configuration of protected mode support macros shall be as mentioned below:

DioRunningInUser0Mode -> false
DioUserModelInitApiEnable -> false
DioUserModeRuntimeApiEnable -> false

9.1.2.1 Configuration of DIO Channel

Each channel with index of type Dio_ChannelType is referenced via symbolic names through the file Dio_Cfg.h.

Code Listing 11 Example for DIO Channel configuration

```
051:  /* User Defined Symbolic Names for the DIO CHANNELS
052:  */
053:  #define MOTOR_START_STOP (DIO_CHANNEL_0_5)
054:  #define MOTOR_DIRECTION  (DIO_CHANNEL_0_8)
055:  #define CAN_TRCV_ENT0     (DIO_CHANNEL_1_1)
056:  #define CAN_TRCV_NSTB0    (DIO_CHANNEL_1_2)
```

9.1.2.2 Configuration of DIO Port

Each port with index of type Dio_PortType is referenced via symbolic names through the file Dio_Cfg.h.

Code Listing 12 Example for DIO Port configuration

```
057:  /* User Defined Symbolic Names for the DIO PORTS
058:  */
059:  #define MOTOR_CTL_PORT   (DIO_PORT_0)
060:  #define CAN_TRCV_PORT    (DIO_PORT_1)
```

9.1.2.3 Configuration of DIO Channel Group

Each channel group which is of type Dio_ChannelGroupType is referenced via symbolic names through the file Dio_Cfg.h.

Code Listing 13 Symbolic Names for the DIO CHANNEL GROUPS

```
061:  /* User Defined Symbolic Names for the DIO CHANNEL GROUPS
062:  */
063:  #define MOTOR_CTL_GRP (&Dio_ChannelGroupConfig[0])
064:  #define CAN_TRCV_GRP  (&Dio_ChannelGroupConfig[1])
```

Dio_PBCfg.c contains the instantiation (=definition) of the DIO configuration structure includes Dio_Cfg.h and uses the defined values for initialization of structure elements.

The user has to select the defined high channel and low channel under the DioPort container to form the required DIO channel Group.

For example, to get the first element of the Dio_ChannelGroupConfig array as in [Code Listing 14](#), the Low channel is referred to the defined DIO channel 'MOTOR_START_STOP' and the High channel is referred to the defined DIO channel 'MOTOR_DIRECTION' under the DioPort container 'MOTOR_CTL_PORT'.

Code Listing 14 Example for DIO Channel Group Data Initialization

```
065:  const Dio_ChannelGroupType Dio_ChannelGroupConfig[2] =
066:  {
067:  {
068:      /* MOTOR_CTL_GRP, MOTOR_CTL_PORT
```

Code Listing 14 Example for DIO Channel Group Data Initialization

```

069:      */
070:      /* mask */      (Dio_PortLevelType) 0x01E0,
071:      /* offset */    (uint8) 5,
072:      /* Port ID */ (Dio_PortType) MOTOR_CTL_PORT
073:  },
074:  {
075:      /* CAN_TRCV_GRP, CAN_TRCV_PORT
076:      */
077:      /* mask */      (Dio_PortLevelType) 0x0006,
078:      /* offset */    (uint8) 1,
079:      /* port ID */ (Dio_PortType) CAN_TRCV_PORT
080:  }
081: };

```

9.2 Initialization of Driver

Initialization of the general purpose I/O pins is not in the scope of DIO driver and is to be done by Port driver.

9.3 Using the API

The [Code Listing 15](#) shows as example calls to different services provided by DIO driver. This code listing uses symbols as described earlier.

Code Listing 15 Using DIO Driver Services

```

082:  Dio_LevelType      ChannelVal;
083:  Dio_PortLevelType  PortVal;
084:  Dio_PortLevelType  ChannelGrpVal;
085:
086:  /* Set level STD_HIGH for port 0 channel 5 */
087:  Dio_WriteChannel(MOTOR_START_STOP, STD_HIGH);
088:
089:  /* Read level of port 0 channel 8 */
090:  ChannelVal = Dio_ReadChannel(MOTOR_DIRECTION);
091:
092:  /* Write port 1 with all pins set to HIGH */
093:  Dio_WritePort(CAN_TRCV_PORT, (Dio_PortLevelType)0x7FFF);
094:
095:  /* Read the level of all the pins of port 0 */
096:  PortVal = Dio_ReadPort(MOTOR_CTL_PORT);
097:
098:  /* Write to channel group 0 */
099:  Dio_WriteChannelGroup(MOTOR_CTL_GRP, (Dio_PortLevelType)0xA);
0100:
0101:  /* Read from channel group 1 */
0102:  ChannelGrpVal = Dio_ReadChannelGroup(CAN_TRCV_GRP);

```

10 Assumptions and Limitations

10.1 Assumptions and Deviations from the Software Specification

- 1) The file structure given in SWS depicts to include Dio_Cbk.h, SchM_Dio.h. SchM_Dio.h is not included as there are no exclusive areas used by Driver. Refer Bugzillas 26854, 26853, 15984.
- 2) Deviation from requirement **DIO149_Conf**: To define the symbolic name for the DIO channel group, the configuration parameter **DioChannelGroupIdentification** will be used, instead of the container name **DioChannelGroup**.
- 3) When configuring the Dio with two Dio Configurations, the respective **DioChannelType** is a pointer to constant unlike uint (Deviation from requirement **DIO182**).

10.2 Limitations and Deviations

None.

10.3 Hardware Features Not Supported

None

11 Used AURIX Derivate

11.1 TC27x

11.1.1 TC2Dx

Table 84 Portpin details for TC2Dx

Derivative	Information	Feature	Availability
TC2Dx	Number of Ports: 18 Number of I/O Pins: 177 Number of input only pins: 10	Port 00	Available pins: 0 – 12
		Port 01	Available pins: 0 – 13
		Port 02	Available pins: 0 – 15
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0 – 8
		Port 11	Available pins: 0 – 15
		Port 12	Available pins: 0 – 1
		Port 13	Available pins: 0 – 3
		Port 14	Available pins: 0 – 10
		Port 15	Available pins: 0 – 8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0 – 3, 6 - 14
		Port 21	Available pins: 0 – 7
		Port 22	Available pins: 0 – 11
		Port 23	Available pins: 0 – 7
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Available pins: 0 – 7
		Port 31	Not available
		Port 32	Available pins: 0 – 6
		Port 33	Available pins: 0 – 15
		Port 34	Available pins: 0 – 10

		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0 – 9

11.1.2 TC277

Table 85 Portpin details for TC277

Derivative	Information	Feature	Availability
TC277	Number of Ports: 17 Number of I/O Pins: 160 Number of input only pins: 10	Port 00	Available pins: 0 – 12
		Port 01	Available pins: 3 – 7
		Port 02	Available pins: 0 – 11
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0 – 8
		Port 11	Available pins: 0 – 15
		Port 12	Available pins: 0 – 1
		Port 13	Available pins: 0 – 3
		Port 14	Available pins: 0 – 10
		Port 15	Available pins: 0 – 8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0 – 3, 6 – 14
		Port 21	Available pins: 0 – 7
		Port 22	Available pins: 0 – 11
		Port 23	Available pins: 0 – 7
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Available pins: 0, 2 – 7

		Port 33	Available pins: 0 – 15
		Port 34	Available pins: 1 – 5
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0 – 9

11.1.3 TC275

Table 86 Portpin details of TC275

Derivative	Information	Feature	Availability
TC275	Number of Ports: 14 Number of I/O Pins: 121 Number of input only pins: 10	Port 00	Available pins: 0 – 12
		Port 01	Not available
		Port 02	Available pins: 0 – 8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0 – 8
		Port 11	Available pins: 2-3, 6, 9-12
		Port 12	Not available
		Port 13	Available pins: 0 – 3
		Port 14	Available pins: 0 – 10
		Port 15	Available pins: 0 – 8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0 – 3, 6 – 14
		Port 21	Available pins: 0 – 7
		Port 22	Available pins: 0 – 3
		Port 23	Available pins: 0 – 5
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available

	Port 31	Not available
	Port 32	Available pins: 0, 2 – 4
	Port 33	Available pins: 0 – 13
	Port 34	Not available
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0 – 9

11.2 TC26x

11.2.1 TC264

Table 87 Portpin details for TC264

Derivative	Information	Feature	Availability
TC264	Number of Ports: 14 Number of I/O Pins: 93 Number of input only pins: 6	Port 00	Available pins: 0-9,12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 1-3,5-6
		Port 11	Available pins: 2-3,6,9-12
		Port 12	Not available
		Port 13	Available pins:0-3
		Port 14	Available pins: 0-6
		Port 15	Available pins: 0-8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0,2-3,6-14
		Port 21	Available pins: 2-7
		Port 22	Available pins: 0-3
		Port 23	Available pins: 1
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available

	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Not available
	Port 31	Not available
	Port 32	Available pins: 0,4
	Port 33	Available pins: 4-13
	Port 34	Not available
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-1,6-9

11.2.2 TC265

Table 88 Portpin details of TC265

Derivative	Information	Feature	Availability
TC265	Number of Ports: 14 Number of I/O Pins: 119 Number of input only pins: 8	Port 00	Available pins: 0-12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0-8
		Port 11	Available pins: 2-3,6,9-12
		Port 12	Not available
		Port 13	Available pins:0-3
		Port 14	Available pins: 0-10
		Port 15	Available pins: 0-8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0-3,6-14
		Port 21	Available pins: 0-7
		Port 22	Available pins: 0-3
		Port 23	Available pins: 0-5
		Port 24	Not available

		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Available pins: 0,2-4
		Port 33	Available pins: 0-13
		Port 34	Not available
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0-9

11.2.3 TC267

Table 89 Portpin details for TC267

Derivative	Information	Feature	Availability
TC267	Number of Ports: 14 Number of I/O Pins: 119 Number of input only pins: 10	Port 00	Available pins: 0-12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0-8
		Port 11	Available pins: 2-3,6,9-12
		Port 12	Not available
		Port 13	Available pins:0-3
		Port 14	Available pins: 0-10
		Port 15	Available pins: 0-8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0-3,6-14
		Port 21	Available pins: 0-7
		Port 22	Available pins: 0-3

		Port 23	Available pins: 0-5
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Available pins: 0,2-4
		Port 33	Available pins: 0-13
		Port 34	Not available
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0-9

11.3 TC29x

11.3.1 TC297

Table 90 Portpin details for TC297

Derivative	Information	Feature	Availability
TC297	Number of Ports: 17 Number of I/O Pins: 160 Number of input only pins: 10	Port 00	Available pins: 0-12
		Port 01	Available pins: 3-7
		Port 02	Available pins: 0-11
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0-8
		Port 11	Available pins: 0-15
		Port 12	Available pins: 0 and 1
		Port 13	Available pins:0-3
		Port 14	Available pins: 0-10
		Port 15	Available pins: 0-8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available

	Port 19	Not available
	Port 20	Available pins: 0-3, 6-14
	Port 21	Available pins: 0-7
	Port 22	Available pins: 0-11
	Port 23	Available pins: 0-7
	Port 24	Not Available
	Port 25	Not Available
	Port 26	Not Available
	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Not Available
	Port 31	Not Available
	Port 32	Available pins: 0,2-7
	Port 33	Available pins: 0-15
	Port 34	Available pins: 1-5
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-9

11.3.2 TC298

Table 91 Portpin details for TC298

Derivative	Information	Feature	Availability
TC298	Number of Ports: 22 Number of I/O Pins: 243 Number of input only pins: 12	Port 00	Available pins: 0-15
		Port 01	Available pins: 0-15
		Port 02	Available pins: 0-15
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0-11, 13-15
		Port 11	Available pins: 0-15
		Port 12	Available pins: 0 and 1
		Port 13	Available pins: 0-7, 9-12, 14
		Port 14	Available pins: 0-15
		Port 15	Available pins: 1-8
		Port 16	Not available

	Port 17	Not available
	Port 18	Not available
	Port 19	Not available
	Port 20	Available pins: 0 and 2
	Port 21	Available pins: 0-7
	Port 22	Available pins: 0-3
	Port 23	Available pins: 0-6
	Port 24	Available pins: 0-15
	Port 25	Available pins: 0-5, 7-15
	Port 26	Available pins: 1
	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Available pins: 0-15
	Port 31	Available pins: 0-15
	Port 32	Available pins: 0,2-7
	Port 33	Available pins: 0-15
	Port 34	Available pins: 1-5
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-7, 11-14

11.3.3 TC299

Table 92 Portpin details for TC299

Derivative	Information	Feature	Availability
TC299	Number of Ports: 22 Number of I/O Pins: 276 Number of input only pins: 15	Port 00	Available pins: 0-15
		Port 01	Available pins: 0-15
		Port 02	Available pins: 0-15
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 0-11, 13-15
		Port 11	Available pins: 0-15
		Port 12	Available pins: 0 and 1
		Port 13	Available pins: 0-7, 9-15
		Port 14	Available pins: 0-15

	Port 15	Available pins: 0-8, 10-15
	Port 16	Not available
	Port 17	Not available
	Port 18	Not available
	Port 19	Not available
	Port 20	Available pins: 0-3,6-14
	Port 21	Available pins: 0-7
	Port 22	Available pins: 0-11
	Port 23	Available pins: 0-7
	Port 24	Available pins: 0-15
	Port 25	Available pins: 0-15
	Port 26	Available pins: 1
	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Available pins: 0-15
	Port 31	Available pins: 0-15
	Port 32	Available pins: 0,2-7
	Port 33	Available pins: 0-15
	Port 34	Available pins: 1-5
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-14

11.4 TC23x

11.4.1 TC233

Table 93 Portpin details for TC233

Derivative	Information	Feature	Availability
TC233	Number of Ports: 13 Number of I/O Pins: 78	Port 00	Available pins: 0
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 5-6

		Port 11	Available pins: 2,3,6,8-12
		Port 12	Not available
		Port 13	Available pins: 0-3
		Port 14	Available pins: 0,1,3,4,6
		Port 15	Available pins: 0,1,2,3,5
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 2,8,9,10,11,12,13,14
		Port 21	Available pins: 2,3,4,6,7
		Port 22	Not available
		Port 23	Available pins: 1
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Not available
		Port 33	Available pins: 5-10
		Port 34	Not available
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0-11
		Port 41	Available pins: 0-11

11.4.2 TC234

Table 94 Portpin details for TC234

Derivative	Information	Feature	Availability
TC234	Number of Ports: 15 Number of I/O Pins: 120	Port 00	Available pins: 0-9,12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available

	Port 08	Not available
	Port 09	Not available
	Port 10	Available pins: 1-3,5-6
	Port 11	Available pins: 2,3,6,8-12
	Port 12	Not available
	Port 13	Available pins: 0-3
	Port 14	Available pins: 0-8
	Port 15	Available pins: 0-8
	Port 16	Not available
	Port 17	Not available
	Port 18	Not available
	Port 19	Not available
	Port 20	Available pins: 0,2,3,6-14
	Port 21	Available pins: 2-7
	Port 22	Available pins: 0-4
	Port 23	Available pins: 1
	Port 24	Not available
	Port 25	Not available
	Port 26	Not available
	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Not available
	Port 31	Not available
	Port 32	Not available
	Port 33	Available pins: 0-12
	Port 34	Available pins: 0-3
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-11
	Port 41	Available pins: 0-11

11.4.3 TC237

Table 95 Portpin details for TC237

Derivative	Information	Feature	Availability
TC237	Number of Ports: 15 Number of I/O Pins: 120	Port 00	Available pins: 0-9,12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available

	Port 05	Not available
	Port 06	Not available
	Port 07	Not available
	Port 08	Not available
	Port 09	Not available
	Port 10	Available pins: 1-3,5-6
	Port 11	Available pins: 2,3,6,8-12
	Port 12	Not available
	Port 13	Available pins: 0-3
	Port 14	Available pins: 0-8
	Port 15	Available pins: 0-8
	Port 16	Not available
	Port 17	Not available
	Port 18	Not available
	Port 19	Not available
	Port 20	Available pins: 0,2,3,6-14
	Port 21	Available pins: 2-7
	Port 22	Available pins: 0-4
	Port 23	Available pins: 1
	Port 24	Not available
	Port 25	Not available
	Port 26	Not available
	Port 27	Not available
	Port 28	Not available
	Port 29	Not available
	Port 30	Not available
	Port 31	Not available
	Port 32	Not available
	Port 33	Available pins: 0-12
	Port 34	Available pins: 0-3
	Port 35	Not available
	Port 36	Not available
	Port 37	Not available
	Port 38	Not available
	Port 39	Not available
	Port 40	Available pins: 0-11
	Port 41	Available pins: 0-11

11.5 TC22x/TC21x

11.5.1 TC222/TC212

Table 96 PortPin details for TC222/TC212

Derivative	Information	Feature	Availability
------------	-------------	---------	--------------

CONFIDENTIAL
Used AURIX Derivate

TC222/TC212	Number of Ports: 12 Number of I/O Pins: 59	Port 00	Available pins: 0
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 5-6
		Port 11	Available pins: 2,3,6,9-12
		Port 12	Not available
		Port 13	Not available
		Port 14	Available pins: 0-1,3-4,6
		Port 15	Available pins: 0-3,5
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 2,8-9,11-14
		Port 21	Available pins: 6-7
		Port 22	Not available
		Port 23	Available pins: 1
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Not available
		Port 33	Available pins: 5-10
		Port 34	Not available
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 5-11
		Port 41	Available pins: 0-6

11.5.2 TC223/TC213

Table 97 PortPin details for TC223/TC213

Derivative	Information	Feature	Availability
TC223/TC213	Number of Ports: 13 Number of I/O Pins: 78	Port 00	Available pins: 0
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 5-6
		Port 11	Available pins: 2-3,6-12
		Port 12	Not available
		Port 13	Available pins: 0-3
		Port 14	Available pins: 0-1,3-4,6
		Port 15	Available pins: 0-3,5
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 2,8-14
		Port 21	Available pins: 2-4,6-7
		Port 22	Not available
		Port 23	Available pins: 1
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Not available
		Port 33	Available pins: 5-10
		Port 34	Not available
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available
		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0-11

		Port 41	Available pins: 0-11
--	--	---------	----------------------

11.5.3 TC224/TC214

Table 98 PortPin details for TC224/TC214

Derivative	Information	Feature	Availability
TC224/TC214	Number of Ports: 15 Number of I/O Pins: 120	Port 00	Available pins: 0-9,12
		Port 01	Not available
		Port 02	Available pins: 0-8
		Port 03	Not available
		Port 04	Not available
		Port 05	Not available
		Port 06	Not available
		Port 07	Not available
		Port 08	Not available
		Port 09	Not available
		Port 10	Available pins: 1-3,5-6
		Port 11	Available pins: 2-3,6,8-12
		Port 12	Not available
		Port 13	Available pins: 0-3
		Port 14	Available pins: 0-8
		Port 15	Available pins: 0-8
		Port 16	Not available
		Port 17	Not available
		Port 18	Not available
		Port 19	Not available
		Port 20	Available pins: 0,2-3,6-14
		Port 21	Available pins: 2-7
		Port 22	0-4
		Port 23	Available pins: 1
		Port 24	Not available
		Port 25	Not available
		Port 26	Not available
		Port 27	Not available
		Port 28	Not available
		Port 29	Not available
		Port 30	Not available
		Port 31	Not available
		Port 32	Not available
		Port 33	Available pins: 0-12
		Port 34	Available pins: 0-3
		Port 35	Not available
		Port 36	Not available
		Port 37	Not available

		Port 38	Not available
		Port 39	Not available
		Port 40	Available pins: 0-11
		Port 41	Available pins: 0-11

www.infineon.com